Unit 14. Convex Optimisation, Support Vector Machines, Neural Networks

1 Convex Optimisation

Prerequisites §§P.3 and P.3b (convexity and separation; convex programming and duality: the theorems **weak duality**, **Slater implies strong duality** and **Karush–Kuhn–Tucker**). The convex optimisation exercises are from the solution manual of Boyd and Vandenberghe, **Convex Optimization** (2004), chapters 5, 7, 8 and 9. This unit is outside the examination syllabus.

1.1 Convex problems

Definition 14.1 (Convex problem)**.** A **convex problem** is

$$\text{minimise } f_0(\mathbf{x}) \quad \text{subject to} \quad f_i(\mathbf{x}) \leqslant 0,\ i = 1, \dots, \ell, \qquad \mathbf{x} \in \mathcal{D}, \tag{1}$$

with $\mathcal{D} \subseteq \mathbb{R}^d$ convex and $f_0, \dots, f_\ell$ convex on $\mathcal{D}$, the problem of the prerequisites' §P.3b. An affine equality $\mathbf{a}'\mathbf{x} = b$ is the pair of convex inequalities $\mathbf{a}'\mathbf{x} - b \leqslant 0$ and $b - \mathbf{a}'\mathbf{x} \leqslant 0$. A **linear programme** has $f_0, \dots, f_\ell$ affine; a **quadratic programme** has $f_0(\mathbf{x}) = \frac{1}{2}\mathbf{x}'\mathbf{P}\mathbf{x} + \mathbf{q}'\mathbf{x}$ with $\mathbf{P}$ positive semi-definite and $f_1, \dots, f_\ell$ affine.

Proposition 14.2 (Local minima are global)**.** In a convex problem the feasible set $\mathcal{F}$ is convex, and a feasible $\mathbf{x}^\star$ that minimises f_0 over $\mathcal{F} \cap \{\mathbf{x} : |\mathbf{x} - \mathbf{x}^\star| < r\}$ for some $r > 0$ minimises it over $\mathcal{F}$. If f_0 is strictly convex there is at most one minimiser.

Proof. If $\mathbf{x}, \mathbf{y} \in \mathcal{F}$ and $\theta \in [0,1]$ then $\theta\mathbf{x} + (1-\theta)\mathbf{y} \in \mathcal{D}$ and $f_i(\theta\mathbf{x} + (1-\theta)\mathbf{y}) \leqslant \theta f_i(\mathbf{x}) + (1-\theta) f_i(\mathbf{y}) \leqslant 0$, so $\mathcal{F}$ is convex. Suppose $\mathbf{y} \in \mathcal{F}$ had $f_0(\mathbf{y}) < f_0(\mathbf{x}^\star)$. For small $\theta > 0$ the point $\mathbf{z} = \mathbf{x}^\star + \theta(\mathbf{y} - \mathbf{x}^\star)$ lies in $\mathcal{F}$ within distance r of $\mathbf{x}^\star$, and $f_0(\mathbf{z}) \leqslant (1-\theta) f_0(\mathbf{x}^\star) + \theta f_0(\mathbf{y}) < f_0(\mathbf{x}^\star)$, a contradiction. If $\mathbf{x}^\star \neq \mathbf{y}^\star$ were two minimisers of a strictly convex f_0, their midpoint would be feasible with a smaller value. □

Example 14.1 (The dual of a quadratic programme with affine constraints)**.** Let $\mathbf{P}$ be positive definite and consider minimising $\tfrac{1}{2}\mathbf{x}'\mathbf{P}\mathbf{x} + \mathbf{q}'\mathbf{x}$ subject to $\mathbf{G}\mathbf{x} \leqslant \mathbf{h}$ (rowwise). The Lagrangian $L(\mathbf{x}, \boldsymbol{\lambda}) = \tfrac{1}{2}\mathbf{x}'\mathbf{P}\mathbf{x} + \mathbf{q}'\mathbf{x} + \boldsymbol{\lambda}'(\mathbf{G}\mathbf{x} - \mathbf{h})$ is strictly convex in $\mathbf{x}$ and minimised where its gradient $\mathbf{P}\mathbf{x} + \mathbf{q} + \mathbf{G}'\boldsymbol{\lambda}$ vanishes, so the dual function is

$$g(\boldsymbol{\lambda}) = -\tfrac{1}{2}(\mathbf{q} + \mathbf{G}'\boldsymbol{\lambda})'\mathbf{P}^{-1}(\mathbf{q} + \mathbf{G}'\boldsymbol{\lambda}) - \mathbf{h}'\boldsymbol{\lambda},$$

a concave quadratic, as the prerequisites' proposition **concavity of the dual** guarantees. The dual problem maximises it over $\boldsymbol{\lambda} \geqslant \mathbf{0}$, a quadratic programme with simple bounds, and $\mathbf{x}^\star = -\mathbf{P}^{-1}(\mathbf{q} + \mathbf{G}'\boldsymbol{\lambda}^\star)$. When some $\tilde{\mathbf{x}}$ has $\mathbf{G}\tilde{\mathbf{x}} < \mathbf{h}$, Slater's condition holds and the two optimal values agree.

1.2 Descent methods

Definition 14.3 (Gradient descent and Newton's method)**.** For a differentiable $f : \mathbb{R}^d \to \mathbb{R}$, **gradient descent** with step $t > 0$ iterates $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - t\nabla f(\mathbf{x}^{(k)})$. For a twice differentiable f with positive definite Hessian, **Newton's method** iterates $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \{\nabla^2 f(\mathbf{x}^{(k)})\}^{-1}\nabla f(\mathbf{x}^{(k)})$, the minimiser of the second-order Taylor approximation of f at $\mathbf{x}^{(k)}$. In practice the step along either direction is shortened by **backtracking**: start from $t = 1$ and multiply by $\beta \in (0,1)$ until f has decreased by at least αt times the absolute value of the directional derivative, $\alpha \in (0, \tfrac{1}{2})$.

Theorem 14.4 (Linear convergence of gradient descent)**.** Let f be twice continuously differentiable on $\mathbb{R}^d$ with $m\mathbf{I} \preccurlyeq \nabla^2 f(\mathbf{x}) \preccurlyeq M\mathbf{I}$ for all $\mathbf{x}$, where $0 < m \leqslant M$, and let $p^\star = \inf f$. Then f

has a unique minimiser, and gradient descent with $t = 1/M$ satisfies

$$f(\mathbf{x}^{(k)}) - p^\star \;\leqslant\; \Big(1 - \frac{m}{M}\Big)^k \big\{f(\mathbf{x}^{(0)}) - p^\star\big\}. \tag{2}$$

Proof. Fix $\mathbf{x}, \mathbf{y}$ and put $\phi(s) = f(\mathbf{x} + s(\mathbf{y} - \mathbf{x}))$. The prerequisites' theorem **Taylor with Lagrange remainder** gives $\phi(1) = \phi(0) + \phi'(0) + \frac{1}{2}\phi''(\sigma)$ for some $\sigma \in (0,1)$, with $\phi''(\sigma) = (\mathbf{y} - \mathbf{x})'\nabla^2 f(\cdot)(\mathbf{y} - \mathbf{x})$ between $m|\mathbf{y} - \mathbf{x}|^2$ and $M|\mathbf{y} - \mathbf{x}|^2$. Hence

$$f(\mathbf{x}) + \nabla f(\mathbf{x})'(\mathbf{y} - \mathbf{x}) + \tfrac{m}{2}|\mathbf{y} - \mathbf{x}|^2 \leqslant f(\mathbf{y}) \leqslant f(\mathbf{x}) + \nabla f(\mathbf{x})'(\mathbf{y} - \mathbf{x}) + \tfrac{M}{2}|\mathbf{y} - \mathbf{x}|^2. \tag{3}$$

A minimiser exists and is unique. The left inequality with $\mathbf{x} = \mathbf{0}$ shows $f(\mathbf{y}) \to \infty$ as $|\mathbf{y}| \to \infty$, so by the prerequisites' lemma **coercive minimisation** f attains its infimum; $\nabla^2 f \succ \mathbf{0}$ makes f strictly convex (the prerequisites' lemma **convexity from the derivative**, along each line), so the minimiser $\mathbf{x}^\star$ is unique (Proposition 14.2).
Decrease. The right inequality with $\mathbf{y} = \mathbf{x} - \nabla f(\mathbf{x})/M$ gives $f(\mathbf{y}) \leqslant f(\mathbf{x}) - |\nabla f(\mathbf{x})|^2/(2M)$.

Suboptimality is bounded by the gradient. Minimising the left side of (3) over $\mathbf{y}$, at $\mathbf{y} = \mathbf{x} - \nabla f(\mathbf{x})/m$, gives $f(\mathbf{y}) \geqslant f(\mathbf{x}) - |\nabla f(\mathbf{x})|^2/(2m)$ for every $\mathbf{y}$, so $p^\star \geqslant f(\mathbf{x}) - |\nabla f(\mathbf{x})|^2/(2m)$, that is $|\nabla f(\mathbf{x})|^2 \geqslant 2m\{f(\mathbf{x}) - p^\star\}$.
Combining the two, $f(\mathbf{x}^{(k+1)}) - p^\star \leqslant f(\mathbf{x}^{(k)}) - p^\star - \frac{m}{M}\{f(\mathbf{x}^{(k)}) - p^\star\}$, and induction gives (2). □

Remark. The rate depends on the condition number M/m of the Hessian. When it is large, gradient descent zigzags across a long narrow valley and needs about $(M/m)\ln(1/\epsilon)$ steps for accuracy ϵ. Newton's method is invariant to linear changes of variable (for $h(\mathbf{y}) = f(\mathbf{A}\mathbf{y})$ with $\mathbf{A}$ invertible, $\nabla h = \mathbf{A}'\nabla f$ and $\nabla^2 h = \mathbf{A}'\nabla^2 f\,\mathbf{A}$, so the Newton step in $\mathbf{y}$ is $\mathbf{A}^{-1}$ times the step in $\mathbf{x} = \mathbf{A}\mathbf{y}$), so the conditioning does not matter to it, and near the minimiser it converges quadratically, the number of correct digits roughly doubling at each step; that theorem (Boyd and Vandenberghe §9.5) is not proved here. Newton's method applied to the negative log likelihood of a logistic regression is the iteratively reweighted least squares of Unit 8. A loss that is not convex has no positive lower bound m on its Hessian, and Theorem 14.4 then says nothing.

2 Support Vector Classifiers

ISLR §9.1–9.2, beyond both reading lists.

Remark (ISLR §9.1)**.** In p dimensions a hyperplane $\beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p = 0$ divides the space in two, and a separating hyperplane classifies an observation by the sign of the left side. If the classes can be separated by a hyperplane there are infinitely many such hyperplanes, and the **maximal margin classifier** chooses the one farthest from the training observations. It can overfit when p is large, and it exists only when the classes are separable.

2.1 The maximal margin classifier

Throughout, the data are $(\mathbf{x}_i, y_i)$ with $\mathbf{x}_i \in \mathbb{R}^p$ and $y_i \in \{-1, +1\}$.

Definition 14.5 (Hyperplane and margins)**.** For $\mathbf{w} \neq \mathbf{0}$ and $b \in \mathbb{R}$ the hyperplane is $\{\mathbf{x} : \mathbf{w}'\mathbf{x} + b = 0\}$ and the classifier is $\operatorname{sign}(\mathbf{w}'\mathbf{x} + b)$. The **functional margin** of observation i is $\hat{\gamma}_i = y_i(\mathbf{w}'\mathbf{x}_i + b)$ and its **geometric margin** is $\gamma_i = \hat{\gamma}_i / |\mathbf{w}|$. The data are **separable** if both labels occur and some $(\mathbf{w}, b)$ has $\hat{\gamma}_i > 0$ for every i.

Proposition 14.6 (The geometric margin is a distance)**.** γ_i is the signed Euclidean distance from $\mathbf{x}_i$ to the hyperplane, positive when $\mathbf{x}_i$ is classified correctly. It is unchanged when $(\mathbf{w}, b)$ is replaced by $(c\mathbf{w}, cb)$ for $c > 0$, while $\hat{\gamma}_i$ is multiplied by c.

Proof. The point of the hyperplane nearest $\mathbf{x}_i$ is $\mathbf{x}_i - t\mathbf{w}$ with $\mathbf{w}'(\mathbf{x}_i - t\mathbf{w}) + b = 0$ (the prerequisites' theorem **orthogonal projection**, applied to the subspace $\{\mathbf{v} : \mathbf{w}'\mathbf{v} = 0\}$ of directions of the hyperplane after translating by a point of it: the difference $t\mathbf{w}$ is orthogonal to that subspace), so $t = (\mathbf{w}'\mathbf{x}_i + b)/|\mathbf{w}|^2$ and the distance is $|t|\,|\mathbf{w}| = |\mathbf{w}'\mathbf{x}_i + b|/|\mathbf{w}|$. Multiplying by y_i attaches the sign. The scaling claims are immediate, $|c\mathbf{w}| = c|\mathbf{w}|$ for $c > 0$. □

Definition 14.7 (Maximal margin problem)**.** Among separating hyperplanes, maximise the smallest geometric margin:

$$\max_{\mathbf{w}\neq\mathbf{0},\, b} \; \min_{i} \; \frac{y_i(\mathbf{w}'\mathbf{x}_i + b)}{|\mathbf{w}|}. \tag{4}$$

Theorem 14.8 (Canonical form)**.** Suppose the data are separable. Then (4) has the same solutions, up to the positive scaling of Proposition 14.6, as the convex programme

$$\min_{\mathbf{w},b} \ \tfrac{1}{2}|\mathbf{w}|^2 \qquad \text{subject to} \qquad y_i(\mathbf{w}'\mathbf{x}_i + b) \geqslant 1, \quad i = 1, \dots, n, \tag{5}$$

and the maximised margin is $1/|\mathbf{w}^*|$, so the two margin boundaries are $2/|\mathbf{w}^*|$ apart.

Proof. By Proposition 14.6 the objective of (4) is invariant under $(\mathbf{w}, b) \mapsto (c\mathbf{w}, cb)$, $c > 0$, so the maximisation may be restricted to any one representative per ray. Choose the representative with $\min_i \hat{\gamma}_i = 1$, possible because the data are separable and $\min_i \hat{\gamma}_i > 0$ can be scaled to 1. On that set the objective is $1/|\mathbf{w}|$, so maximising it is minimising $|\mathbf{w}|$, equivalently $\frac{1}{2}|\mathbf{w}|^2$, subject to $\min_i \hat{\gamma}_i \geqslant 1$, which is (5).

The constraint $\min_i \hat{\gamma}_i \geqslant 1$ is binding at the optimum, since otherwise $(\mathbf{w}, b)$ could be divided by $\min_i \hat{\gamma}_i > 1$, reducing the objective while staying feasible. Hence $\min_i \gamma_i = 1/|\mathbf{w}^*|$ and the slab between the two margin boundaries has width twice that. □

Theorem 14.9 (Existence and uniqueness)**.** If the data are separable, (5) has exactly one solution $(\mathbf{w}^*, b^*)$.

Proof. The feasible set is an intersection of closed half-spaces, hence closed and convex, and it is non-empty by separability and Theorem 14.8. The objective $\frac{1}{2}|\mathbf{w}|^2$ is continuous and coercive in $\mathbf{w}$ on that set, and b is confined to a bounded range once $\mathbf{w}$ is bounded, because for any i with $y_i = +1$ and j with $y_j = -1$ the constraints give $1 - \mathbf{w}'\mathbf{x}_i \leqslant b \leqslant -1 - \mathbf{w}'\mathbf{x}_j$. So the minimisation may be taken over a compact set, the feasible points with $|\mathbf{w}| \leqslant |\mathbf{w}_0|$ for a feasible $(\mathbf{w}_0, b_0)$, and the infimum is attained (the prerequisites' theorem **uniform continuity on compacts**).
For uniqueness, $\mathbf{w} \mapsto \frac{1}{2}|\mathbf{w}|^2$ is strictly convex, so two solutions must share the same $\mathbf{w}^*$; and given $\mathbf{w}^*$ the feasible b form an interval on which the objective is constant, so every feasible b is optimal. Each optimal b makes some constraint tight (Theorem 14.8) and so is an endpoint of the interval, and an interval all of whose points are endpoints is a single point. □

Remark. Nothing in the argument mentions a probability model. The maximal margin classifier is a geometric object defined by an optimisation problem, and (5) is exactly the form the prerequisites' duality theory takes as input: a convex objective and affine constraints.

2.2 The dual problem

Theorem 14.10 (The dual of the maximal margin problem)**.** For separable data the Lagrangian dual of (5) is

$$\max_{\boldsymbol{\alpha}\geqslant\mathbf{0}} \sum_{i=1}^{n}\alpha_i - \tfrac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n}\alpha_i\alpha_j y_i y_j\, \mathbf{x}_i'\mathbf{x}_j \qquad \text{subject to} \qquad \sum_i \alpha_i y_i = 0, \tag{6}$$

strong duality holds, and at the optimum

$$\mathbf{w}^* = \sum_{i=1}^{n}\alpha_i^* y_i \mathbf{x}_i, \qquad |\mathbf{w}^*|^2 = \sum_i \alpha_i^*. \tag{7}$$

Proof. The Lagrangian of (5) with multipliers $\alpha_i \geqslant 0$ is

$$L(\mathbf{w}, b, \boldsymbol{\alpha}) = \tfrac{1}{2}|\mathbf{w}|^2 - \sum_i \alpha_i\{y_i(\mathbf{w}'\mathbf{x}_i + b) - 1\},$$

which is convex in $(\mathbf{w}, b)$, so its infimum is where the gradient vanishes: $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$ from $\partial/\partial \mathbf{w}$, and $\sum_i \alpha_i y_i = 0$ from $\partial/\partial b$. If $\sum_i \alpha_i y_i \neq 0$ the infimum over b is $-\infty$, so the dual function is finite only on that hyperplane. Substituting,

$$\inf_{\mathbf{w},b} L = \tfrac{1}{2}\Big|\sum_i \alpha_i y_i \mathbf{x}_i\Big|^2 - \Big|\sum_i \alpha_i y_i \mathbf{x}_i\Big|^2 + \sum_i \alpha_i = \sum_i \alpha_i - \tfrac{1}{2}\sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i' \mathbf{x}_j,$$

which is the objective of (6). Maximising the dual function is the dual problem by the prerequisites' definition.

Strong duality: the problem is feasible and its value is finite, and doubling a feasible $(\mathbf{w}, b)$ makes every constraint strict, $y_i(\mathbf{w}'\mathbf{x}_i + b) \geqslant 2 > 1$, so Slater's condition holds and the prerequisites' theorem **Slater implies strong duality** applies.

The first identity of (7) is the stationarity condition just derived, evaluated at $(\mathbf{w}^*, b^*, \boldsymbol{\alpha}^*)$, where it holds by the stationarity condition of the prerequisites' theorem **Karush–Kuhn–Tucker**, which applies by strong duality and Theorem 14.9, which supplies the primal optimum. For the second, strong duality equates the primal value $\frac{1}{2}|\mathbf{w}^*|^2$ with the dual value $\sum_i \alpha_i^* - \frac{1}{2}|\mathbf{w}^*|^2$, using $\sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i' \mathbf{x}_j = |\mathbf{w}^*|^2$; rearranging gives $\sum_i \alpha_i^* = |\mathbf{w}^*|^2$. □

Corollary 14.11 (Support vectors)**.** At the optimum, complementary slackness gives

$$\alpha_i^* \{ y_i(\mathbf{w}^{*\prime}\mathbf{x}_i + b^*) - 1 \} = 0 \qquad \text{for every } i. \tag{8}$$

So $\alpha_i^* > 0$ only for observations with functional margin exactly 1, that is lying on a margin boundary; the observations on a margin boundary are the **support vectors**, and every $\alpha_i^* > 0$ belongs to one of them. By Theorem 14.10, $\mathbf{w}^*$ is a combination of the support vectors alone, and the classifier

$$\operatorname{sign}\Big(\sum_{i \in \mathrm{SV}} \alpha_i^* y_i \, \mathbf{x}_i^\prime \mathbf{x} + b^* \Big) \tag{9}$$

does not depend on the other observations at all. The margin width is $2/\sqrt{\sum_i \alpha_i^*}$.

Proof. (8) is the complementary slackness condition of the prerequisites' Karush–Kuhn–Tucker theorem, whose hypotheses hold here by the strong duality just established and the primal optimum of Theorem 14.9. If the constraint is slack then $\alpha_i^* = 0$, so only tight constraints carry weight;

substituting (7) into $\mathbf{w}'\mathbf{x}+b$ gives (9), and the margin statement combines Theorem 14.8 with the second half of (7). □

Remark. That the solution depends on a handful of observations is the substantive content, and it is not a design choice but a consequence of complementary slackness. Moving a point that is not a support vector, so long as it stays outside the slab on its own side, does not change the classifier at all: the old $(\mathbf{w}^*, b^*, \boldsymbol{\alpha}^*)$ still satisfies the Karush–Kuhn–Tucker conditions for the moved data, so by the converse half of the prerequisites' theorem it stays optimal, and by Theorem 14.9 its $(\mathbf{w}^*, b^*)$ is the only solution of (5). Contrast least squares, where every observation enters $\hat{\boldsymbol{\beta}}$ through $\mathbf{X}'\mathbf{Y}$.

2.3 Non-separable data

Definition 14.12 (Soft margin)**.** For $C > 0$,

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \ \tfrac{1}{2}|\mathbf{w}|^2 + C\sum_i \xi_i \qquad \text{subject to} \qquad y_i(\mathbf{w}'\mathbf{x}_i + b) \geqslant 1 - \xi_i, \quad \xi_i \geqslant 0. \tag{10}$$

Theorem 14.13 (Soft-margin dual and the box constraint)**.** The dual of (10) is (6) with the single change that $\boldsymbol{\alpha}$ is confined to the box $0 \leqslant \alpha_i \leqslant C$. Complementary slackness gives, at a primal optimum $(\mathbf{w}^*, b^*, \boldsymbol{\xi}^*)$ and a dual optimum $\boldsymbol{\alpha}^*$, with α_i standing for α_i^* and $\hat{\gamma}_i$ for $y_i(\mathbf{w}^{*\prime}\mathbf{x}_i + b^*)$,

$$\alpha_i = 0 \;\Rightarrow\; \hat{\gamma}_i \geqslant 1, \qquad 0 < \alpha_i < C \;\Rightarrow\; \hat{\gamma}_i = 1, \qquad \alpha_i = C \;\Rightarrow\; \hat{\gamma}_i \leqslant 1. \tag{11}$$

Proof. With multipliers $\alpha_i \geqslant 0$ for the margin constraints and $\mu_i \geqslant 0$ for $\xi_i \geqslant 0$, the Lagrangian is $\frac{1}{2}|\mathbf{w}|^2 + C\sum_i \xi_i - \sum_i \alpha_i\{y_i(\mathbf{w}'\mathbf{x}_i + b) - 1 + \xi_i\} - \sum_i \mu_i\xi_i$. Stationarity in $\mathbf{w}$ and b gives the same two conditions as before, and stationarity in ξ_i gives $C - \alpha_i - \mu_i = 0$. Since $\mu_i \geqslant 0$ this is $\alpha_i \leqslant C$, and substituting removes ξ from the dual function, leaving the objective of (6).
Complementary slackness needs strong duality and a primal and a dual optimum (the prerequisites' theorem **Karush–Kuhn–Tucker**). The point $(\mathbf{w}, b, \boldsymbol{\xi}) = (\mathbf{0}, 0, 2{\cdot}\mathbf{1})$ makes every constraint strict and the optimal value is finite (it is at least 0), so **Slater implies strong duality** gives $d^\star = p^\star$ and a dual optimum. For a primal optimum, the best ξ_i for fixed $(\mathbf{w}, b)$ is $\max\{0, 1 - \hat{\gamma}_i\}$, which leaves the continuous function $F(\mathbf{w}, b) = \frac{1}{2}|\mathbf{w}|^2 + C\sum_i \max\{0, 1 - y_i(\mathbf{w}'\mathbf{x}_i + b)\}$ on $\mathbb{R}^{p+1}$. If both labels occur, F is coercive: $F \geqslant \frac{1}{2}|\mathbf{w}|^2$, and for bounded $\mathbf{w}$ the term of an i with $y_i = +1$ or of

a j with $y_j = -1$ grows without bound as $b \to -\infty$ or $b \to +\infty$. So the prerequisites' lemma **coercive minimisation** gives a minimiser $(\mathbf{w}^*, b^*)$, and with $\xi_i^* = \max\{0, 1 - \hat{\gamma}_i\}$ a primal optimum. (If only one label occurs, $(\mathbf{0}, y_1, \mathbf{0})$ attains the lower bound 0.)
For (11): $\alpha_i < C$ forces $\mu_i > 0$, hence $\xi_i = 0$ by complementary slackness on the second constraint, hence $\hat{\gamma}_i \geqslant 1$; and $\alpha_i > 0$ forces the first constraint tight, $\hat{\gamma}_i = 1 - \xi_i \leqslant 1$. Combining the two gives the three cases. □

Proposition 14.14 (Hinge loss)**.** Problem (10) is equivalent to the unconstrained

$$\min_{\mathbf{w},b} \sum_{i=1}^{n} \max\{0,\ 1 - y_i(\mathbf{w}'\mathbf{x}_i + b)\} + \frac{1}{2C}|\mathbf{w}|^2, \tag{12}$$

a convex penalised loss of the shape Unit 6 studied, with the hinge in place of squared error and $1/(2C)$ in place of λ.

Proof. For fixed $(\mathbf{w}, b)$ the constraints of (10) say $\xi_i \geqslant \max\{0, 1-\hat{\gamma}_i\}$, and the objective increases in ξ_i, so the optimum takes ξ_i equal to that lower bound. Substituting and dividing by C gives (12). Convexity: a maximum of two affine functions is convex, and $|\mathbf{w}|^2$ is convex. □

Remark. So C is a regularisation parameter read backwards. Large C penalises violations heavily, gives a narrow margin and typically few support vectors; small C gives a wide margin, typically many support vectors at the bound, and a smaller $|\mathbf{w}|$. The last claim is a theorem. Let $C_1 < C_2$, let $(\mathbf{w}_k, b_k)$ minimise (12) at C_k, write $\lambda_k = 1/(2C_k)$, $W_k = |\mathbf{w}_k|^2$ and h_k for the hinge sum at $(\mathbf{w}_k, b_k)$. Optimality at C_1 and at C_2 gives $h_1 + \lambda_1 W_1 \leqslant h_2 + \lambda_1 W_2$ and $h_2 + \lambda_2 W_2 \leqslant h_1 + \lambda_2 W_1$; adding, $(\lambda_1 - \lambda_2)(W_1 - W_2) \leqslant 0$, and $\lambda_1 > \lambda_2$ gives $W_1 \leqslant W_2$. The support-vector counts are not monotone in C in general, hence "typically".

Remark (ISLR §9.2)**.** ISLR writes the support vector classifier with a **budget**: maximise the margin M subject to $y_i(\beta_0 + \boldsymbol{\beta}'\mathbf{x}_i) \geqslant M(1-\epsilon_i)$, $\epsilon_i \geqslant 0$, $\sum_i \epsilon_i \leqslant C$ and $|\boldsymbol{\beta}| = 1$, its (9.12)–(9.15). Its C is a bound on the total violation, so a larger C allows more violations and gives a wider margin; the C of Definition 14.12, which is `e1071`'s `cost`, multiplies the violations and works the other way. Either way the observations on the margin or on its wrong side are the support vectors,

and the classifier does not depend on the others, unlike linear discriminant analysis, which uses every observation through the class means and covariance.

3 Support Vector Machines

ISLR §9.3–9.5, beyond both reading lists.

Remark (ISLR §9.3)**.** A linear boundary in the original features can be made non-linear by enlarging the feature space, with squares, cubes and products of the features, and fitting a support vector classifier there. The **support vector machine** does this implicitly, through a kernel, so the enlarged space never has to be formed.

3.1 Kernels

Definition 14.15 (Kernel)**.** A **kernel** is a symmetric $K : \mathbb{R}^p \times \mathbb{R}^p \to \mathbb{R}$ that is **positive semi-definite**: for every finite set $\mathbf{x}_1, \ldots, \mathbf{x}_n$ the Gram matrix $\mathbf{K} = [K(\mathbf{x}_i, \mathbf{x}_j)]$ is positive semi-definite.

Proposition 14.16 (A feature map gives a kernel, and makes the dual concave)**.** If $K(\mathbf{x},\mathbf{z}) = \varphi(\mathbf{x})'\varphi(\mathbf{z})$ for some $\varphi : \mathbb{R}^p \to \mathbb{R}^d$ then K is a kernel. For any kernel, the dual objective of (6) with $\mathbf{x}_i'\mathbf{x}_j$ replaced by $K(\mathbf{x}_i,\mathbf{x}_j)$ is concave in $\boldsymbol{\alpha}$, so (6) remains a convex programme and the prerequisites' duality theory still applies.

Proof. For a feature map, $\mathbf{c}'\mathbf{K}\mathbf{c} = \sum_{i,j} c_i c_j \varphi(\mathbf{x}_i)'\varphi(\mathbf{x}_j) = \left|\sum_i c_i \varphi(\mathbf{x}_i)\right|^2 \geqslant 0$.
For concavity, write the quadratic term as $\boldsymbol{\alpha}'\mathbf{Q}\boldsymbol{\alpha}$ with $\mathbf{Q} = \operatorname{diag}(\mathbf{y})\mathbf{K}\operatorname{diag}(\mathbf{y})$. For any $\mathbf{c}$, $\mathbf{c}'\mathbf{Q}\mathbf{c} = \mathbf{d}'\mathbf{K}\mathbf{d}$ with $d_i = y_i c_i$, which is non-negative because $\mathbf{K}$ is positive semi-definite. So $\mathbf{Q}$ is positive semi-definite, $-\frac{1}{2}\boldsymbol{\alpha}'\mathbf{Q}\boldsymbol{\alpha}$ is concave, and adding the linear term $\sum_i \alpha_i$ preserves concavity. □

Example 14.2 (The quadratic kernel)**.** For $p = 2$, $K(\mathbf{x},\mathbf{z}) = (1 + \mathbf{x}'\mathbf{z})^2$ equals $\varphi(\mathbf{x})'\varphi(\mathbf{z})$ with

$$\varphi(\mathbf{x}) = \left(1,\ \sqrt{2}x_1,\ \sqrt{2}x_2,\ x_1^2,\ x_2^2,\ \sqrt{2}x_1x_2\right)',$$

as expanding $(1 + x_1z_1 + x_2z_2)^2$ shows. Fitting (6) with this K is fitting a linear machine in the six features without ever forming them.

Remark. The dual (6) and the classifier (9) involve the data only through inner products, which makes the substitution legitimate. For a kernel with no finite feature map the kernel machine is still a linear machine: given the training points and any query point $\mathbf{x}$, the Gram matrix of K on these $n+1$ points is positive semi-definite, so by the spectral theorem it equals $\mathbf{F}\mathbf{F}'$ with $\mathbf{F} = \mathbf{U}\mathbf{\Lambda}^{1/2}$, where $\mathbf{U}\mathbf{\Lambda}\mathbf{U}'$ is its spectral decomposition, and the rows $\mathbf{f}_1, \dots, \mathbf{f}_n, \mathbf{f}_\mathbf{x}$ of $\mathbf{F}$ are finite-dimensional features with $\mathbf{f}_i'\mathbf{f}_j = K(\mathbf{x}_i, \mathbf{x}_j)$ and $\mathbf{f}_i'\mathbf{f}_\mathbf{x} = K(\mathbf{x}_i, \mathbf{x})$. The dual of the maximal margin or support vector classifier on $\mathbf{f}_1, \dots, \mathbf{f}_n$ (Theorem 14.13, or Theorem 14.10 when these features are separable) involves the features only through these inner products, so its $\boldsymbol{\alpha}^*$ is that of the kernel dual; b^* is not a dual variable, but it is fixed by complementary slackness, (8) or (11), which also uses only the values $K(\mathbf{x}_j, \mathbf{x}_i)$; and by (7) its decision value at $\mathbf{f}_\mathbf{x}$ is $\sum_i \alpha_i^* y_i K(\mathbf{x}_i, \mathbf{x}) + b^*$. The radial kernel $K(\mathbf{x}, \mathbf{z}) = \exp(-\sigma|\mathbf{x} - \mathbf{z}|^2)$, $\sigma > 0$, is a kernel. Since $|\mathbf{x} - \mathbf{z}|^2 = |\mathbf{x}|^2 + |\mathbf{z}|^2 - 2\mathbf{x}'\mathbf{z}$, $K(\mathbf{x}, \mathbf{z}) = e^{-\sigma|\mathbf{x}|^2} e^{-\sigma|\mathbf{z}|^2} \sum_{k \geqslant 0} (2\sigma)^k (\mathbf{x}'\mathbf{z})^k / k!$. Each $(\mathbf{x}'\mathbf{z})^k = \varphi_k(\mathbf{x})'\varphi_k(\mathbf{z})$, where $\varphi_k(\mathbf{x})$ lists the p^k products $x_{j_1} \cdots x_{j_k}$, so it is a kernel by Proposition 14.16. A sum of positive semi-definite Gram matrices with non-negative weights is positive semi-definite, so is a limit of such matrices ($\mathbf{c}'\mathbf{K}\mathbf{c} \geqslant 0$ passes to the limit), and so is $\mathbf{D}\mathbf{K}\mathbf{D}$ for the diagonal $\mathbf{D}$ with $D_{ii} = e^{-\sigma|\mathbf{x}_i|^2}$. The corresponding φ is infinite-dimensional (not proved here), so the primal (5) cannot be written down in the feature

space of φ (on the training points it can, through $\mathbf{K} = \mathbf{F}\mathbf{F}'$) while the dual is an ordinary finite quadratic programme in n variables, so the dual is the object one solves.

Remark (ISLR §9.3)**.** ISLR's kernels are the polynomial kernel of degree d, $K(\mathbf{x}, \mathbf{z}) = (1+\mathbf{x}'\mathbf{z})^d$ (a kernel by Proposition 14.16, with φ listing the weighted monomials of the multinomial expansion, as in Example 14.2), and the radial kernel $K(\mathbf{x}, \mathbf{z}) = \exp(-\gamma|\mathbf{x} - \mathbf{z}|^2)$, and its fitted function is $f(\mathbf{x}) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i K(\mathbf{x}, \mathbf{x}_i)$, its (9.23), a sum over the support vectors, which is (9) with the kernel in place of the inner product. The radial kernel is local: an observation far from $\mathbf{x}$ contributes almost nothing to $f(\mathbf{x})$.

Remark (ISLR §9.4)**.** With $M > 2$ classes there are two schemes. **One-versus-one** fits a classifier to each of the $\binom{M}{2}$ pairs of classes and assigns an observation to the class it is most often assigned to. **One-versus-all** fits M classifiers, each class against the rest, and assigns the class whose fitted $f_k(\mathbf{x})$ is largest. `e1071` uses one-versus-one.

Remark (ISLR §9.5)**.** By Proposition 14.14 the support vector classifier minimises the hinge loss $\sum_i \max\{0, 1 - y_i f(\mathbf{x}_i)\}$ plus a ridge penalty $\lambda|\mathbf{w}|^2$, ISLR's (9.25)–(9.26). The logistic regression

loss $\sum_i \ln\{1 + e^{-y_i f(\mathbf{x}_i)}\}$ is close to it: both are convex, small for observations well on the right side and roughly linear far on the wrong side. The hinge loss is exactly 0 once $y_i f(\mathbf{x}_i) \geqslant 1$, which is why only the support vectors matter; the logistic loss is never exactly 0. ISLR's advice: the support vector machine when the classes are well separated, logistic regression when they overlap; and kernels can be used with logistic regression too, though historically they were used mostly with support vector machines.

4 Neural Networks

ISLR §10.1–10.2, §10.3 and §10.5 (overview), §10.6–10.8, beyond both reading lists. Prerequisites §P.11 (matrix differentiation and the chain rule).

Remark (ISLR §10.1)**.** A neural network with a single hidden layer builds H derived features $A_k = g(w_{k0} + \sum_j w_{kj} X_j)$ from the inputs and fits a linear model in them, $f(X) = \beta_0 + \sum_k \beta_k A_k$, ISLR's (10.1)–(10.2). The derived features are learned, not fixed in advance as they are for a basis expansion. The activation g is the sigmoid $1/(1+e^{-z})$ or, now more often, the rectified linear unit $\max(0, z)$.

4.1 The multilayer perceptron

Definition 14.17 (One hidden layer)**.** With activation $g : \mathbb{R} \to \mathbb{R}$ applied coordinatewise, weights $\mathbf{W}_1 \in \mathbb{R}^{H\times p}$, $\mathbf{w}_2 \in \mathbb{R}^H$ and offsets $\mathbf{c}_1 \in \mathbb{R}^H$, $c_2 \in \mathbb{R}$, the network is

$$\mathbf{z}(\mathbf{x}) = \mathbf{W}_1\mathbf{x} + \mathbf{c}_1, \qquad \mathbf{a}(\mathbf{x}) = g(\mathbf{z}(\mathbf{x})), \qquad f(\mathbf{x}) = \mathbf{w}_2'\mathbf{a}(\mathbf{x}) + c_2. \tag{13}$$

The parameters are $\boldsymbol{\theta} = (\mathbf{W}_1, \mathbf{c}_1, \mathbf{w}_2, c_2)$, of dimension $Hp + 2H + 1$, and the fit minimises $L(\boldsymbol{\theta}) = \frac{1}{2n}\sum_i \{y_i - f(\mathbf{x}_i)\}^2$.

Remark. With H arbitrary and g the identity, f is linear in $\mathbf{x}$ and (13) is the model of Unit 2 with a redundant parametrisation: nothing is gained, and the composition of linear maps is linear however many layers are stacked. The activation is the whole of the difference.

Definition 14.18 (Softmax output and cross-entropy; ISLR (10.13)–(10.14))**.** For a qualitative response with classes $0, \dots, M-1$ a network has M outputs $Z_0, \dots, Z_{M-1}$, linear in the last

hidden layer, and models

$$\Pr(Y = m \mid X) = f_m(X) = \frac{e^{Z_m}}{\sum_{\ell=0}^{M-1} e^{Z_\ell}}, \tag{14}$$

fitted by minimising the **cross-entropy** $-\sum_i \sum_m y_{im} \ln f_m(\mathbf{x}_i)$, with $y_{im} = 1$ if observation i is in class m and 0 otherwise: the negative multinomial log likelihood.

Remark (ISLR §10.2)**.** A **multilayer** network feeds the derived features of one hidden layer into the next. ISLR's digit recognition network (`MNIST`: 28×28 images, 10 classes) has two hidden layers of 256 and 128 units and a softmax output, about 235,000 weights for 60,000 training images, so regularisation is essential.

Definition 14.19 (Convolution layer; ISLR §10.3)**.** A **convolution filter** of size $q \times q$ on an $N \times N$ image computes, at each of the $(N - q + 1)^2$ positions where it fits inside the image, the sum of the q^2 pixel values times the filter's q^2 weights, plus a bias, and applies the activation. A

convolution layer applies several filters, each giving one **feature map**. A **pooling** layer then summarises each non-overlapping 2×2 block of a feature map by its maximum.

Remark (ISLR §10.3–10.4)**.** A convolution layer is a hidden layer whose units see only a small patch of the image and share their weights across positions, so the same feature is detected wherever it occurs and the number of weights is that of the filters, not of the image. ISLR's convolutional network for the `CIFAR100` images, and its document classifier for the `IMDb` reviews, need data and pretrained models not used here.

4.2 Backpropagation

Theorem 14.20 (The gradient)**.** Let g be differentiable (for the rectifier $\max(0, u)$, which is not differentiable at 0, the identities below hold at every $\boldsymbol{\theta}$ with no $z_{ih} = 0$, and software uses them everywhere with the convention $g'(0) = 0$). Write $\mathbf{Z} = \mathbf{X}\mathbf{W}_1' + \mathbf{1}\mathbf{c}_1'$ and $\mathbf{A} = g(\mathbf{Z})$, both $n \times H$, and $\mathbf{f} = \mathbf{A}\mathbf{w}_2 + c_2\mathbf{1}$. Put $\mathbf{d}_2 = (\mathbf{f} - \mathbf{y})/n \in \mathbb{R}^n$ and

$$\mathbf{D}_1 = (\mathbf{d}_2\mathbf{w}_2') \odot g'(\mathbf{Z}) \ \in \mathbb{R}^{n \times H}, \tag{15}$$

$\odot$ being entrywise multiplication. Then

$$\frac{\partial L}{\partial \mathbf{w}_2} = \mathbf{A}'\mathbf{d}_2, \qquad \frac{\partial L}{\partial c_2} = \mathbf{1}'\mathbf{d}_2, \qquad \frac{\partial L}{\partial \mathbf{W}_1} = \mathbf{D}_1'\mathbf{X}, \qquad \frac{\partial L}{\partial \mathbf{c}_1} = \mathbf{D}_1'\mathbf{1}. \tag{16}$$

Proof. $L = \frac{1}{2n}|\mathbf{y} - \mathbf{f}|^2$, so $\partial L/\partial \mathbf{f} = (\mathbf{f} - \mathbf{y})/n = \mathbf{d}_2$ by the prerequisites' rule for the differential of a quadratic form.

Since $\mathbf{f} = \mathbf{A}\mathbf{w}_2 + c_2\mathbf{1}$ with $\mathbf{A}$ not depending on $\mathbf{w}_2$ or c_2, the chain rule (prerequisites) gives $\partial L/\partial \mathbf{w}_2 = \mathbf{A}'\mathbf{d}_2$ and $\partial L/\partial c_2 = \mathbf{1}'\mathbf{d}_2$, which are the first two identities.

For the first layer, f_i depends on z_{ih} only through $a_{ih} = g(z_{ih})$ and $\partial f_i/\partial a_{ih} = w_{2h}$, so $\partial L/\partial z_{ih} = d_{2i}w_{2h}g'(z_{ih})$, which is entry (i,h) of (15). Then $z_{ih} = \mathbf{w}_{1h}'\mathbf{x}_i + c_{1h}$ gives $\partial z_{ih}/\partial w_{1hj} = x_{ij}$ and $\partial z_{ih}/\partial c_{1h} = 1$, so summing over i,

$$\frac{\partial L}{\partial w_{1hj}} = \sum_i D_{1,ih}x_{ij}, \qquad \frac{\partial L}{\partial c_{1h}} = \sum_i D_{1,ih},$$

which are the last two identities in matrix form. □

Remark. The name records the shape of the computation: $\mathbf{d}_2$ is formed at the output and pushed back through $\mathbf{w}_2$ and g' to give $\mathbf{D}_1$, and each layer's two derivatives are then one matrix product. The cost is a small constant multiple of that of one forward pass, whatever the number of parameters (an operation count not proved here), so (16) and not a difference quotient is what is computed. All twenty-six partial derivatives of a small network agree with central differences to 10^{-10}.

4.3 What one hidden layer can represent

Theorem 14.21 (Rectified units and piecewise linearity)**.** Let $p = 1$ and $g(u) = \max(0, u)$. Then

(i) every network (13) with H units is continuous and piecewise linear with breakpoints among the points $-c_{1h}/w_{1h}$ with $w_{1h} \neq 0$ (a unit with $w_{1h} = 0$ is constant and contributes no breakpoint), hence has at most $H + 1$ linear pieces;

(ii) conversely every continuous piecewise-linear f with breakpoints $t_1 < \cdots < t_H$ ($H \geqslant 1$) is exactly representable by such a network with $H + 1$ units.

Proof. (i) Each term $w_{2h}\max(0, w_{1h}x + c_{1h})$ with $w_{1h} \neq 0$ is continuous, is affine on each side of $-c_{1h}/w_{1h}$, and has no other kink, and a term with $w_{1h} = 0$ is constant; a finite sum of such functions is continuous and affine on each interval between consecutive kinks. With H kinks the line is cut into at most $H + 1$ intervals.
(ii) Let f have slope s_0 on $(-\infty, t_1)$ and slope s_h on (t_h, t_{h+1}). Put $w_{1h} = 1$, $c_{1h} = -t_h$ and $w_{2h} = s_h - s_{h-1}$ for $h = 1, \dots, H$. The candidate $\tilde{f}(x) = c_2 + s_0 x + \sum_h w_{2h}\max(0, x - t_h)$ has slope s_0 below t_1 and gains $s_h - s_{h-1}$ at each t_h, hence has the slopes of f everywhere; choosing c_2 to match f at one point makes $\tilde{f} = f$, two continuous functions with equal derivatives on a connected set differing by a constant. The linear term $s_0 x$ is absorbed by one more unit at t_1: $s_0 x = s_0 t_1 + s_0\max(0, x - t_1) - s_0\max(0, t_1 - x)$, so w_{21} becomes s_1, the extra unit has weight -1, offset t_1 and output weight $-s_0$, and $s_0 t_1$ joins c_2. □

Remark. So, by Theorem 14.21, a one-dimensional rectified network is precisely a piecewise-linear regression with free knots, and H counts the knots, plus one unit for the slope at $-\infty$. That is a modest class, and the reason such networks are useful is not expressiveness at fixed H but that

H may be taken large and the knots are fitted rather than chosen. Compare Unit 11: a tree is piecewise **constant** with knots fitted greedily one at a time.

4.4 Why the optimisation is different

Proposition 14.22 (Permutation symmetry)**.** Let P be a permutation of $\{1,\dots,H\}$ and let $\boldsymbol{\theta}^P$ permute the rows of $\mathbf{W}_1$, the entries of $\mathbf{c}_1$ and the entries of $\mathbf{w}_2$ by P. Then f is unchanged, so $L(\boldsymbol{\theta}^P) = L(\boldsymbol{\theta})$. Consequently every minimiser with distinct hidden units belongs to an orbit of $H!$ minimisers. Moreover, for $g(u) = \max(0,u)$ the loss L is not convex, for every $H \geqslant 1$.

Proof. $\mathbf{w}_2' g(\mathbf{W}_1\mathbf{x} + \mathbf{c}_1) = \sum_h w_{2h} g(\mathbf{w}_{1h}'\mathbf{x} + c_{1h})$ is a sum, unchanged by reordering its terms. For non-convexity, set every parameter to 0 except $c_{11} = w_{21} = s$ and $c_2 = \bar{y} - 1$. Then $f \equiv \bar{y} - 1 + s\max(0,s)$ and, since $\sum_i (y_i - \bar{y}) = 0$, $L(s) = V + \frac{1}{2}\{1 - s\max(0,s)\}^2$ with $V = \frac{1}{2n}\sum_i (y_i - \bar{y})^2$. So $L(-1) = L(0) = V + \frac{1}{2}$ and $L(1) = V$, and $L(0) > \frac{1}{2}\{L(-1) + L(1)\}$: L is not convex along this line. □

Remark. The support vector machine of (5) is a convex programme, so its solution is unique (Theorem 14.9), the duality theory of the prerequisites applies exactly, and any algorithm that stops at a stationary point has found the answer. Nothing of this survives for (13): there are at least $H!$ global minima whenever L has a minimiser with distinct hidden units, no duality, and gradient descent returns whatever basin it started in.

Definition 14.23 (Gradient descent and its stochastic form)**.** Gradient descent iterates $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla L(\boldsymbol{\theta})$ with step size $\eta > 0$, ∇L being given by Theorem 14.20 (with $g'(0) = 0$ for the rectifier). The **stochastic** form replaces ∇L by the same expression computed on a random subset of the observations, which is unbiased for ∇L when the subset is drawn uniformly.

Proposition 14.24 (Unbiasedness)**.** Let $\mathcal{B}$ be a uniformly random subset of $\{1, \dots, n\}$ of size m and $L_{\mathcal{B}}(\boldsymbol{\theta}) = \frac{1}{2m} \sum_{i \in \mathcal{B}} \{y_i - f(\mathbf{x}_i)\}^2$. Then $\mathsf{E}[\nabla L_{\mathcal{B}}(\boldsymbol{\theta})] = \nabla L(\boldsymbol{\theta})$ for every fixed $\boldsymbol{\theta}$.

Proof. $\nabla L_{\mathcal{B}} = \frac{1}{m} \sum_{i \in \mathcal{B}} \mathbf{g}_i$ with $\mathbf{g}_i$ the gradient of the ith term, and each i belongs to $\mathcal{B}$ with probability m/n, so the expectation is $\frac{1}{m} \cdot \frac{m}{n} \sum_i \mathbf{g}_i = \nabla L$. □

Remark. Unbiasedness of the step is all that is claimed. It is not a convergence theorem: for a non-convex L none is available at this level, and the practice of restarting, of shrinking η, and of stopping on a validation set is a response to exactly that absence.

Remark (ISLR §10.7)**.** ISLR fits a network by gradient descent with the gradient computed by backpropagation (Theorem 14.20), in its stochastic form on small **minibatches**, and controls overfitting three ways: a ridge or lasso penalty on the weights; **early stopping**, at the epoch where a validation error is smallest; and **dropout**, which at each step sets a random fraction ϕ of the hidden units to zero and scales the others by $1/(1-\phi)$, so no unit can rely on another. The number of layers and units, the dropout rate, the penalty and the minibatch size are tuning parameters.

Remark (ISLR §10.6, §10.8)**.** Deep learning works best when the training set is very large and the signal-to-noise ratio high, as with images and text; on a small tabular data set ISLR's network on `Hitters` does no better than least squares or the lasso, which are simpler and interpretable, and Occam's razor prefers them. ISLR's §10.8 adds **double descent**: as the number of parameters passes the number of observations, the test error of a minimum-norm interpolating fit can fall again

after its peak, because among the many interpolating solutions the fitting method picks a smooth one. This does not contradict the bias–variance trade-off of Unit 1, which concerns a fixed family indexed by flexibility.

4.5 Recurrent networks

Definition 14.25 (Recurrent network; ISLR §10.5)**.** The input is a sequence $X_1, \dots, X_T$ of vectors in $\mathbb{R}^p$. With weights $\mathbf{W} \in \mathbb{R}^{H\times p}$, $\mathbf{U} \in \mathbb{R}^{H\times H}$, $\mathbf{w}_0 \in \mathbb{R}^H$, $\boldsymbol{\beta} \in \mathbb{R}^H$ and β_0, a **recurrent network** computes

$$\mathbf{A}_\ell = g(\mathbf{w}_0 + \mathbf{W}X_\ell + \mathbf{U}\mathbf{A}_{\ell-1}), \qquad \ell = 1, \dots, T, \quad \mathbf{A}_0 = \mathbf{0}, \tag{17}$$

and the output $O_T = \beta_0 + \boldsymbol{\beta}'\mathbf{A}_T$. The same $\mathbf{W}$, $\mathbf{U}$ and $\mathbf{w}_0$ are used at every step.

Remark (ISLR §10.5.2)**.** ISLR's forecaster predicts the day's log trading volume on the New York Stock Exchange from the previous $T = 5$ days of the three standardised series of Unit 10's example on `NYSE`, with $H = 12$ hidden units: each training sequence has $T = 5$ vectors $X_\ell \in \mathbb{R}^3$. Its test R^2 is 0.42, against 0.41 for the linear autoregression of Unit 10 on the fifteen lagged values. With

g the identity, unrolling (17) makes O_T linear in $X_1, \ldots, X_T$ with coefficient vectors $\boldsymbol{\beta}'\mathbf{U}^{T-\ell}\mathbf{W}$: a linear autoregression with constrained coefficients.

5 Computation in R

ISLR §9.6 (support vector machines) and §10.9.1, §10.9.6 (networks), beyond both reading lists. The networks are fitted with `torch` directly; ISLR's labs use `keras` or `luz`, which are not used here, so its network numbers are matched only up to the randomness of stochastic gradient descent. The `MNIST`, `CIFAR100` and `IMDb` labs §10.9.2–10.9.5 need downloaded data and pretrained models and are not reproduced.

5.1 The ISLR labs

Example 14.3 (The support vector classifier; ISLR §9.6.1)**.** Twenty observations in two classes, the second shifted by $(1, 1)$, ISLR's seed:

```
suppressMessages({library(ISLR2); library(e1071)})
set.seed(1); x9 <- matrix(rnorm(20 * 2), ncol = 2); y9 <- c(rep(-1, 10), rep(1, 10))
x9[y9 == 1, ] <- x9[y9 == 1, ] + 1; d9 <- data.frame(x = x9, y = as.factor(y9))
```

```r
sapply(c(cost_10 = 10, cost_0.1 = 0.1), function(C)
  length(svm(y ~ ., d9, kernel = "linear", cost = C, scale = FALSE)$index))
```

```
##  cost_10 cost_0.1
##        7       16
```

```r
set.seed(1)
tu9 <- tune(svm, y ~ ., data = d9, kernel = "linear",
            ranges = list(cost = c(0.001, 0.01, 0.1, 1, 5, 10, 100)))
c(best_cost = tu9$best.parameters$cost, cv_error = tu9$best.performance)
```

```
## best_cost  cv_error
##      0.10      0.05
```

```
xt9 <- matrix(rnorm(20 * 2), ncol = 2); yt9 <- sample(c(-1, 1), 20, rep = TRUE)
xt9[yt9 == 1, ] <- xt9[yt9 == 1, ] + 1; dt9 <- data.frame(x = xt9, y =
 ↪  as.factor(yt9))
table(predict = predict(tu9$best.model, dt9), truth = dt9$y)
```

```
##         truth
## predict -1 1
##      -1  9 1
##      1   2 8
```

ISLR's numbers are reproduced: 7 support vectors at `cost` 10 and 16 at 0.1, a smaller penalty giving a wider margin; cross-validation chooses 0.1 with error 0.05; and the chosen classifier puts 17 of the 20 test observations in the right class.

Example 14.4 (The support vector machine; ISLR §9.6.2 and §9.6.5)**.** Two hundred observations with a non-linear boundary, half for training; then ISLR's `Khan` gene expression data, 2308 genes on 63 training and 20 test tissue samples in four classes:

```
set.seed(1); xr <- matrix(rnorm(200 * 2), ncol = 2)
xr[1:100, ] <- xr[1:100, ] + 2; xr[101:150, ] <- xr[101:150, ] - 2
dr <- data.frame(x = xr, y = as.factor(c(rep(1, 150), rep(2, 50))))
```

```r
trr <- sample(200, 100)
set.seed(1)
tur <- tune(svm, y ~ ., data = dr[trr, ], kernel = "radial",
            ranges = list(cost = c(0.1, 1, 10, 100, 1000),
                          gamma = c(0.5, 1, 2, 3, 4)))
unlist(tur$best.parameters)
```

```
##  cost gamma
##   1.0   0.5
```

```r
table(true = dr[-trr, "y"], pred = predict(tur$best.model, newdata = dr[-trr, ]))
```

```
##     pred
## true  1  2
##    1 67 10
##    2  2 21
```

```r
dk <- data.frame(x = Khan$xtrain, y = as.factor(Khan$ytrain))
kh <- svm(y ~ ., data = dk, kernel = "linear", cost = 10)
```

```
c(training_errors = sum(kh$fitted != dk$y), test_errors =
  sum(predict(kh, data.frame(x = Khan$xtest)) != Khan$ytest))
```

```
## training_errors     test_errors
##               0               2
```

Cross-validation chooses `cost` 1 and $\gamma = 0.5$, and 12% of the test observations are misclassified, ISLR's result. On `Khan`, with far more genes than samples, the linear classifier separates the training samples exactly and makes 2 errors on the 20 test samples; with $p \gg n$ a separating hyperplane always exists for samples in general position, as here (an affine function then interpolates any labels), so the training fit says nothing.

Example 14.5 (A single-layer network on `Hitters`; ISLR §10.9.1, Table 10.2)**.** Salary on the 19 standardised predictors, a third of the 263 players held out, ISLR's seed; least squares, the lasso, and a network with 50 rectified units and dropout 0.4, trained by `rmsprop` for 1500 epochs on minibatches of 32:

```
suppressMessages({library(glmnet); library(torch)})
Gitters <- na.omit(Hitters); ng <- nrow(Gitters)
set.seed(13); testid <- sample(1:ng, trunc(ng / 3))
```

```
xg <- scale(model.matrix(Salary ~ . - 1, data = Gitters)); yg <- Gitters$Salary
lpred <- predict(lm(Salary ~ ., data = Gitters[-testid, ]), Gitters[testid, ])
cvg <- cv.glmnet(xg[-testid, ], yg[-testid], type.measure = "mae")
cpred <- predict(cvg, xg[testid, ], s = "lambda.min")
torch_manual_seed(13)
net <- nn_sequential(nn_linear(ncol(xg), 50), nn_relu(), nn_dropout(0.4),
                     nn_linear(50, 1))
fitnet <- function(net, X, Y, epochs, bs, lossf = nnf_mse_loss) {
  opt <- optim_rmsprop(net$parameters, lr = 0.001); m <- X$size(1)
  for (ep in 1:epochs) { net$train(); perm <- sample(m)
    for (s in seq(1, m, by = bs)) { b <- perm[s:min(s + bs - 1, m)]
      opt$zero_grad(); lossf(net(X[b, ..]), Y[b, ])$backward(); opt$step() } }
  net$eval(); net }
tf <- function(a) torch_tensor(a, dtype = torch_float())
net <- fitnet(net, tf(xg[-testid, ]), tf(matrix(yg[-testid])), 1500, 32)
npred <- as.numeric(net(tf(xg[testid, ])))
sc <- function(p) c(mae = mean(abs(yg[testid] - p)),
                    r2 = 1 - mean((yg[testid] - p)^2) /
```

```
                        mean((yg[testid] - mean(yg[testid]))^2))
round(rbind(least_squares = sc(lpred), lasso = sc(cpred), network = sc(npred)), 3)
```

```
##                  mae    r2
## least_squares 254.7 0.561
## lasso         252.3 0.513
## network       246.3 0.568
```

Least squares and the lasso give ISLR's mean absolute errors and test R^2, 254.7 and 0.56, 252.3 and 0.51. The network gives 246.3 and 0.57 in this run against ISLR's 257.4 and 0.54: a network's fit depends on the random start, the minibatches and the dropout masks, and runs differ by about as much as the three methods do. ISLR's conclusion, that the linear fits are as good and far simpler, is Occam's razor.

Example 14.6 (A recurrent network for `NYSE`; ISLR §10.9.6)**.** The forecaster of Definition 14.25 with $T = 5$, $H = 12$ and dropout 0.1 on the output, 200 epochs on minibatches of 64:

```
xd <- scale(data.matrix(NYSE[, c("DJ_return", "log_volume", "log_volatility")]))
L <- 5; idx <- (L + 1):nrow(xd); yv <- xd[idx, "log_volume"]; itr <- NYSE$train[idx]
seqs <- function(Z) { S <- array(0, c(length(idx), L, ncol(Z)))
```

```
  for (l in 1:L) S[, l, ] <- Z[idx - (L - l + 1), ]; S }
Xs <- seqs(xd)
rnnmod <- nn_module(
  initialize = function(p) { self$rnn <- nn_rnn(p, 12, batch_first = TRUE)
    self$drop <- nn_dropout(0.1); self$out <- nn_linear(12, 1) },
  forward = function(x) self$out(self$drop(self$rnn(x)[[1]][, L, ])))
r2v <- function(p) 1 - mean((p - yv[!itr])^2) / var(yv[!itr])
torch_manual_seed(1); set.seed(1)
rn <- fitnet(rnnmod(3), tf(Xs[itr, , ]), tf(matrix(yv[itr])), 200, 64)
c(training_sequences = sum(itr), test_sequences = sum(!itr),
  rnn_test_r2 = r2v(as.numeric(rn(tf(Xs[!itr, , ])))))
```

```
## training_sequences     test_sequences        rnn_test_r2
##          4276.0000          1770.0000             0.4123
```

The test R^2 is 0.41, ISLR's 0.42 up to the randomness of the fit, and that of Unit 10's linear autoregression on the same five lags. Nothing here is gained by the recurrence.

5.2 The simulated data

The quadratic programmes of §2 are solved directly with `quadprog`, which minimises $\frac{1}{2}\mathbf{d}'\mathbf{D}\mathbf{d} - \mathbf{d}_0'\mathbf{d}$ subject to $\mathbf{A}'\mathbf{d} \geqslant \mathbf{b}_0$, and the results are checked against `e1071`. The function `dualfit` solves the

primal (10), with $\mathbf{x}_i'\mathbf{x}_j$ replaced by K_{ij} through a factorisation $\mathbf{K} = \mathbf{F}\mathbf{F}'$, and returns the dual variables α_i as the multipliers of its margin constraints, which is what the prerequisites' theorem **Karush–Kuhn–Tucker** makes them; the dual solved directly on a rank-deficient $\mathbf{K}$ returns points that violate the box $0 \leqslant \alpha_i \leqslant C$.

```
suppressMessages({library(quadprog); library(e1071)})
set.seed(3); n <- 40
X <- matrix(rnorm(n*2), n, 2)
y <- ifelse(X[, 1] + X[, 2] > 0.6, 1, -1)
X[y == 1, ] <- X[y == 1, ] + 0.9
dualfit <- function(K, y, C) { m <- length(y)
  e <- eigen(K, symmetric = TRUE); k <- e$values > 1e-9*e$values[1]
  F <- e$vectors[, k, drop = FALSE] %*% diag(sqrt(e$values[k]), sum(k))
  r <- ncol(F)                          # K = F F'; primal variables (v, b, xi)
  D <- diag(c(rep(1, r), rep(1e-8, m + 1)))
  A <- rbind(cbind(y*F, y, diag(m)), cbind(matrix(0, m, r + 1), diag(m)))
  s <- solve.QP(D, c(rep(0, r + 1), rep(-C, m)), t(A), rep(c(1, 0), each = m))
  pmax(s$Lagrangian[1:m], 0) }        # multipliers of the margin constraints
```

```
c(n = n, positives = sum(y == 1),  # separable: X1 + X2 > 2.4 for y = 1, <= 0.6 for y
  = -1
  separable = min((X[, 1] + X[, 2])[y == 1]) > max((X[, 1] + X[, 2])[y == -1]))
```

```
##         n positives separable
##        40        14          1
```

5.3 The dual and the support vectors (§2.2)

```
Kl <- X %*% t(X); Cbig <- 1e6
al <- dualfit(Kl, y, Cbig)
w <- drop(t(X) %*% (al*y)); sv <- which(al > 1e-6)
b0 <- mean(y[sv] - X[sv, ] %*% w)
c(support_vectors = length(sv), of_n = n,
  constraint_sum_alpha_y = sum(al*y),
  norm_w_squared = sum(w^2), sum_alpha = sum(al),
```

```
  margin_from_w = 2/sqrt(sum(w^2)),
  margin_from_alpha = 2/sqrt(sum(al)))
```

```
##        support_vectors                     of_n
##          3.00000000000         40.00000000000
## constraint_sum_alpha_y         norm_w_squared
##         -0.00000001565          2.18219044302
##              sum_alpha          margin_from_w
##          2.18219046752          1.35389090546
##      margin_from_alpha
##          1.35389089786
```

The two expressions for the margin agree because $|\mathbf{w}^*|^2 = \sum_i \alpha_i^*$, which is the half of (7) that strong duality supplies. Complementary slackness (8) says the functional margin is exactly 1 wherever $\alpha_i > 0$, and primal feasibility that it is at least 1 everywhere.

```
fm <- drop(y*(X %*% w + b0))
c(max_deviation_from_one_at_SV = max(abs(fm[sv] - 1)),
```

```
  smallest_functional_margin = min(fm),
  largest_alpha_off_the_margin = max(al[abs(fm - 1) > 1e-4]))
```

```
## max_deviation_from_one_at_SV   smallest_functional_margin
##                    4.441e-15                    1.000e+00
## largest_alpha_off_the_margin
##                    0.000e+00
```

`e1071` orders the two classes the other way, so its $(\mathbf{w}, b)$ is the negative of the one above and the hyperplane is the same; the row of sums is what should vanish, and does, to about 5×10^{-4}, within the tolerance 10^{-3} of its solver.

```
m1 <- svm(X, factor(y), kernel = "linear", cost = Cbig, scale = FALSE)
wp <- drop(t(m1$coefs) %*% m1$SV)
rbind(manual = c(w = w, b = b0), e1071 = c(wp, -m1$rho),
      sum = c(w + wp, b0 - m1$rho))
```

```
##                 w1         w2          b
## manual   1.0655411  1.0231386 -1.5651147
## e1071   -1.0656645 -1.0226707  1.5647157
## sum     -0.0001233  0.0004679 -0.0003991
```

```r
c(support_vectors_manual = length(sv), e1071 = nrow(m1$SV))
```

```
## support_vectors_manual                  e1071
##                      3                      3
```

5.4 The soft margin (§2.3)

The data are made non-separable by setting six labels to the negatives of six others drawn separately (two labels change).

```r
set.seed(5); n2 <- 60
X2 <- matrix(rnorm(n2*2), n2, 2)
y2 <- ifelse(X2[, 1] + X2[, 2] > 0, 1, -1)
y2[sample(n2, 6)] <- -y2[sample(n2, 6)]
K2 <- X2 %*% t(X2)
t(sapply(c(0.1, 1, 10), function(C) {
  a <- dualfit(K2, y2, C); ww <- drop(t(X2) %*% (a*y2))
  free <- a > 1e-7 & a < C - 1e-7
  bb <- if (any(free)) mean(y2[free] - X2[free, , drop = FALSE] %*% ww) else 0
  g <- drop(y2*(X2 %*% ww + bb))
  pk <- svm(X2, factor(y2), kernel = "linear", cost = C, scale = FALSE)
```

```
  c(C = C, norm_w = sqrt(sum(ww^2)),
    norm_w_e1071 = sqrt(sum(drop(t(pk$coefs) %*% pk$SV)^2)),
    margin_width = 2/sqrt(sum(ww^2)),
    support_vectors = sum(a > 1e-7), at_the_bound = sum(a > C - 1e-7),
    max_margin_of_free_SV = if (any(free)) max(abs(g[free] - 1)) else NA) }))
```

```
##          C norm_w norm_w_e1071 margin_width support_vectors
## [1,]  0.1  1.177        1.177       1.6989              36
## [2,]  1.0  2.270        2.271       0.8811              22
## [3,] 10.0  2.778        2.778       0.7200              18
##      at_the_bound max_margin_of_free_SV
## [1,]           34             2.849e-13
## [2,]           19             4.818e-12
## [3,]           15             1.623e-10
```

$|\mathbf{w}|$ grows with C, so the margin narrows; the number of support vectors falls; and every support vector with $0 < \alpha_i < C$ sits exactly on the margin, as the middle case of (11) in Theorem 14.13 requires.

5.5 Kernels (§3.1)

```r
phi <- function(Z) cbind(1, sqrt(2)*Z[, 1], sqrt(2)*Z[, 2],
                         Z[, 1]^2, Z[, 2]^2, sqrt(2)*Z[, 1]*Z[, 2])
Kq <- (1 + X2 %*% t(X2))^2
c(gram_from_map_vs_kernel = max(abs(phi(X2) %*% t(phi(X2)) - Kq)),
  duals_agree_to = max(abs(dualfit(Kq, y2, 1) -
                           dualfit(phi(X2) %*% t(phi(X2)), y2, 1))),
  smallest_eigenvalue_of_K = min(eigen(Kq, symmetric = TRUE,
                                       only.values = TRUE)$values))
```

```
##  gram_from_map_vs_kernel           duals_agree_to
##                1.421e-14                1.776e-15
## smallest_eigenvalue_of_K
##               -7.953e-14
```

The Gram matrix is positive semi-definite, as Proposition 14.16 requires of any kernel arising from a feature map, so the dual stays a concave maximisation.

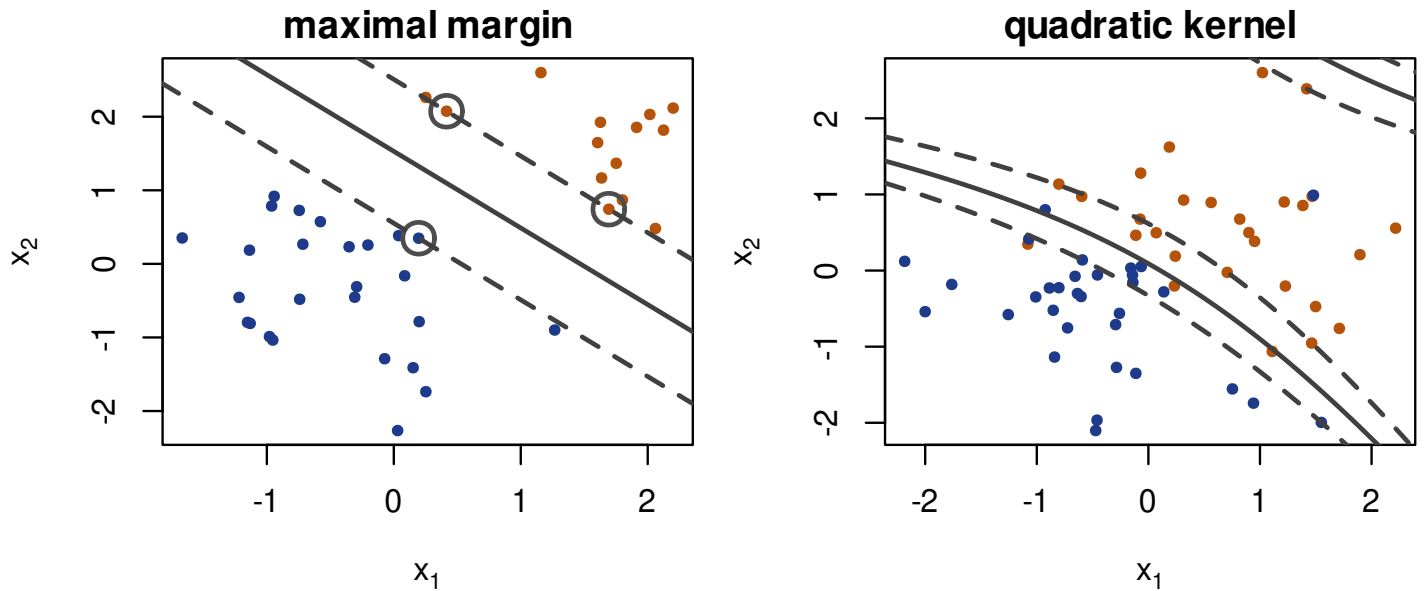


Figure 14.1: Left: the maximal margin hyperplane with its two margin boundaries and the three support vectors circled; by Corollary 14.11 and the remark after it moving any other point, so long as it stays outside the slab on its own side, changes nothing. Right: the same construction with $K(\mathbf{x},\mathbf{z}) = (1+\mathbf{x}'\mathbf{z})^2$ on non-separable data, where the boundary is a conic because the machine is linear in the six features of Example 14.2.

5.6 Backpropagation (§4.2)

```
set.seed(2); nn <- 30; pp <- 3; H <- 5
Xn <- matrix(rnorm(nn*pp), nn, pp); yn <- rnorm(nn)
unpack <- function(th) list(W1 = matrix(th[1:(H*pp)], H, pp),
  c1 = th[H*pp + 1:H], w2 = th[H*pp + H + 1:H], c2 = th[H*pp + 2*H + 1])
sig <- function(z) 1/(1 + exp(-z))
fwd <- function(th) { q <- unpack(th)
  Z <- Xn %*% t(q$W1) + rep(q$c1, each = nn); A <- sig(Z)
  list(A = A, f = drop(A %*% q$w2) + q$c2) }
loss <- function(th) mean((yn - fwd(th)$f)^2)/2
grad <- function(th) { q <- unpack(th); o <- fwd(th)
  d2 <- (o$f - yn)/nn
  D1 <- outer(d2, q$w2) * o$A * (1 - o$A)
  c(as.vector(t(D1) %*% Xn), colSums(D1), drop(t(o$A) %*% d2), sum(d2)) }
th <- rnorm(H*pp + 2*H + 1, sd = 0.5)
num <- sapply(seq_along(th), function(j) { e <- rep(0, length(th))
  e[j] <- 1e-6; (loss(th + e) - loss(th - e))/2e-6 })
```

```
c(parameters = length(th), largest_absolute_gap = max(abs(grad(th) - num)),
  largest_relative_gap = max(abs(grad(th) - num)/(abs(num) + 1e-10)))
```

```
##           parameters largest_absolute_gap
##            2.600e+01             7.266e-11
## largest_relative_gap
##            3.072e-07
```

The derivative of the logistic activation is $g' = g(1-g)$, so `o$A * (1 - o$A)` appears in place of $g'(\mathbf{Z})$ in (15).

```
set.seed(4); Hr <- 6
a1 <- rnorm(Hr); w1 <- rnorm(Hr); w2 <- rnorm(Hr); c2 <- rnorm(1)
fr1 <- function(x) c2 + sum(w2*pmax(0, w1*x + a1))
brk <- sort(-a1/w1)
mid <- c(brk[1] - 1, (brk[-Hr] + brk[-1])/2, brk[Hr] + 1)
slope <- sapply(mid, function(z) (fr1(z + 1e-6) - fr1(z - 1e-6))/2e-6)
c(units = Hr, breakpoints = length(unique(round(brk, 8))),
```

```
  pieces = length(mid), distinct_slopes = length(unique(round(slope, 6))),
  upper_bound_on_pieces = Hr + 1)
```

```
##                 units           breakpoints
##                     6                     6
##                pieces       distinct_slopes
##                     7                     7
## upper_bound_on_pieces
##                     7
```

5.7 Convex against non-convex (§4.4)

```
suppressMessages(library(nnet))
hinge <- function(v, C) { ww <- v[1:2]; bb <- v[3]
  sum(pmax(0, 1 - y2*(X2 %*% ww + bb))) + sum(ww^2)/(2*C) }
svmruns <- sapply(1:10, function(s) { set.seed(s)
  optim(rnorm(3, sd = 3), hinge, C = 1, method = "BFGS",
        control = list(reltol = 1e-14, maxit = 5000))$value })
nnruns <- sapply(1:10, function(s) { set.seed(s)
```

```
  nnet(Xn, yn, size = 5, linout = TRUE, trace = FALSE, maxit = 400)$value })
rbind(hinge_objective = round(svmruns, 5), nnet_objective = round(nnruns, 5))
```

```
##                   [,1]   [,2]   [,3]  [,4]   [,5]   [,6]
## hinge_objective 18.604 18.605 18.605 18.60 18.605 18.605
## nnet_objective   7.556  7.073  4.048  4.24  3.581  4.703
##                   [,7]   [,8]   [,9] [,10]
## hinge_objective 18.605 18.616 18.605 18.62
## nnet_objective   6.777  2.549  9.133  1.46
```

```
c(hinge_spread = diff(range(svmruns)), nnet_spread = diff(range(nnruns)),
  hinge_relative = diff(range(svmruns))/min(svmruns),
  nnet_relative = diff(range(nnruns))/min(nnruns))
```

```
##   hinge_spread    nnet_spread hinge_relative  nnet_relative
##      0.0162020      7.6729238      0.0008709      5.2541112
```

Both are minimised from ten random starting points: the soft-margin objective in its unconstrained hinge form of Proposition 14.14, and the network. The first returns the same value from every start, to within one part in a thousand; the small spread is BFGS stalling at the kinks of an objective that is convex but not differentiable there. The second returns a range of a factor of six. That is the

difference between a convex programme, every local minimum of which is global (Proposition 14.2), and a non-convex one.

5.8 Fitting in practice

Not standardising before a kernel

Every kernel in §3.1 is a function of inner products or of distances, both of which are computed in whatever units the columns carry, exactly as in Unit 13.

```
set.seed(8); n3 <- 200
Z <- cbind(rnorm(n3), 100*rnorm(n3))
lab <- factor(ifelse(Z[, 1] + Z[, 2]/100 > 0, "a", "b"))
err <- function(sc) { i <- 1:100
  m <- svm(Z[i, ], lab[i], kernel = "radial", cost = 1, scale = sc)
  mean(predict(m, Z[-i, ]) != lab[-i]) }
c(unscaled = err(FALSE), scaled = err(TRUE))
```

```
## unscaled   scaled
##     0.37     0.04
```

The symptom is a radial or polynomial kernel whose fit changes when a column is re-expressed in different units. Standardise. `svm` does so by default, so the failure here had to be provoked with `scale = FALSE`: the second column has a hundred times the spread of the first, so $|\mathbf{x}-\mathbf{z}|^2$ is essentially its squared difference alone and the first column is invisible to the kernel.

Reading C as a bound on errors

```
t(sapply(c(0.1, 1, 10, 100), function(C) {
  a <- dualfit(K2, y2, C); ww <- drop(t(X2) %*% (a*y2))
  free <- a > 1e-7 & a < C - 1e-7
  bb <- if (any(free)) mean(y2[free] - X2[free, , drop = FALSE] %*% ww) else 0
  c(C = C, support_vectors = sum(a > 1e-7),
```

```
    training_errors = sum(drop(y2*(X2 %*% ww + bb)) < 0),
    margin = 2/sqrt(sum(ww^2))) }))
```

```
##          C support_vectors training_errors margin
## [1,]   0.1              36               4 1.6989
## [2,]   1.0              22               4 0.8811
## [3,]  10.0              18               4 0.7200
## [4,] 100.0              17               2 0.6770
```

The symptom is a sentence of the form "C controls how many points may be misclassified". By Proposition 14.14 the role of C is that of $1/(2\lambda)$ in a penalised fit, so small C means heavy shrinkage of $\mathbf{w}$, a wide margin and typically many support vectors, and large C the reverse. It bounds nothing: C multiplies the total hinge loss and no individual constraint, so no value of C promises any particular number of mistakes. Here thirty-six of the sixty observations are support vectors at $C = 0.1$, where the margin is widest and four fall on the wrong side, and seventeen at $C = 100$ with two wrong. The count moves with C and is nowhere equal to it.

Counting support vectors as a model-size measure

The symptom is a claim that a machine with few support vectors is simpler, used to choose C. §5.4 shows the count falling as C rises, which is the direction opposite to complexity: large C narrows the margin and fits harder. The count is a property of the geometry, not a penalty, and C is chosen by cross-validation on the prediction error like any other tuning parameter.

Reporting one run of a network

```
suppressMessages(library(nnet))
set.seed(11)
v <- sapply(1:20, function(s) { set.seed(s)
  nnet(Xn, yn, size = 5, linout = TRUE, trace = FALSE, maxit = 400)$value })
```

```
c(best = min(v), worst = max(v), median = median(v),
  distinct_to_six_places = length(unique(round(v, 6))))
```

```
##                    best                   worst
##                   1.234                  13.589
##                  median distinct_to_six_places
##                   4.323                  20.000
```

The symptom is a network reported without saying how many starts were tried. Proposition 14.22 guarantees at least $H! = 120$ global minima here if L has a minimiser with distinct hidden units, and says nothing about local ones, and gradient descent returns the basin it started in. Run from several starts, keep the best by a held-out criterion, and report the number of starts, exactly as Unit 13 requires of K-means, and for the same reason.

Trusting a hand-derived gradient

The symptom is a backpropagation implementation that trains without error and converges slowly to something plausible. Check the gradient against central differences before using it, as §5.6 does. The first version of that section's `grad` returned $\mathbf{X}'\mathbf{D}_1$ where (16) requires $\mathbf{D}_1'\mathbf{X}$, a transposition that leaves

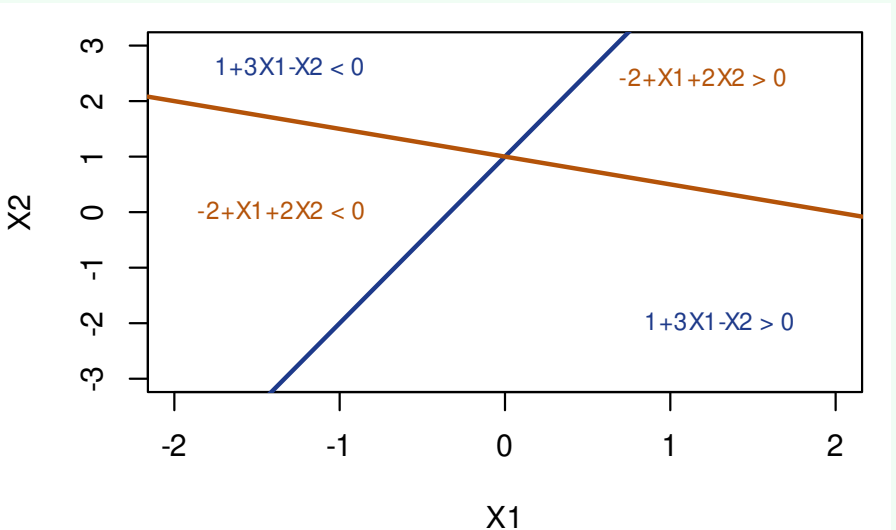
the dimensions of the parameter vector correct and twelve of the fifteen entries of $\partial L/\partial \mathbf{W}_1$ wrong (the other fourteen of the 26 values happen to coincide): the difference check reported a largest gap of 0.03 against 7×10^{-11} after the repair. Nothing in the training loop would have revealed it.

6 Exercises

Exercise 14.1 (ISLR §9.7, exercise 1)**.** (a) Sketch the hyperplane $1 + 3X_1 - X_2 = 0$ and indicate where $1 + 3X_1 - X_2 > 0$ and where it is negative. (b) On the same plot, the same for $-2 + X_1 + 2X_2 = 0$.

Solution.

```r
par(mar = c(4, 4.2, 0.8, 0.8), cex = 0.85)
plot(NA, xlim = c(-2, 2), ylim = c(-3, 3), xlab = "X1", ylab = "X2")
abline(a = 1, b = 3, lwd = 2, col = "#1E3A8A")
abline(a = 1, b = -1/2, lwd = 2, col = "#B45309")
text(c(1.3, -1.3), c(-2, 2.6), c("1+3X1-X2 > 0", "1+3X1-X2 < 0"),
     col = "#1E3A8A", cex = 0.8)
text(c(1.2, -1.35), c(2.4, 0), c("-2+X1+2X2 > 0", "-2+X1+2X2 < 0"),
     col = "#B45309", cex = 0.8)
```



(a) The line $X_2 = 1+3X_1$; $1+3X_1-X_2 > 0$ below it (to the lower right) and < 0 above it. (b) The line $X_2 = 1-X_1/2$; $-2+X_1+2X_2 > 0$ above it and < 0 below it. A hyperplane in two dimensions is a line, and the sign of the left side says which side of it a point is on (Proposition 14.6). ■

Exercise 14.2 (ISLR §9.7, exercise 2)**.** (a), (b) Sketch the curve $(1+X_1)^2+(2-X_2)^2 = 4$ and the regions where the left side is > 4 and $\leqslant 4$. (c) A classifier assigns blue when $(1+X_1)^2+(2-X_2)^2 > 4$

and red otherwise; classify $(0,0)$, $(-1,1)$, $(2,2)$, $(3,8)$. (d) Show that the boundary, not linear in X_1, X_2, is linear in X_1, X_1^2, X_2, X_2^2.

Solution. (a), (b) The curve is the circle of radius 2 about $(-1,2)$; the left side exceeds 4 outside it and is at most 4 on and inside it. (c) The values are $1+4=5$, $0+1=1$, $9+0=9$ and $16+36=52$, so $(0,0)$, $(2,2)$ and $(3,8)$ are blue and $(-1,1)$ is red. (d) Expanding, $(1+X_1)^2+(2-X_2)^2-4 = 1+2X_1+X_1^2-4X_2+X_2^2$, an affine function of (X_1, X_1^2, X_2, X_2^2): the boundary is a hyperplane in that enlarged feature space, the idea of §3. ■

Exercise 14.3 (ISLR §9.7, exercise 3)**.** Seven observations: red $(3,4)$, $(2,2)$, $(4,4)$, $(1,4)$; blue $(2,1)$, $(4,3)$, $(4,1)$. (a)–(c) Find the optimal separating hyperplane and the classification rule; (d), (e) the margin and the support vectors; (f) argue that a slight movement of the seventh observation does not change the hyperplane; (g) give a separating hyperplane that is not optimal; (h) add a point that makes the classes inseparable.

Solution. The problem (5) solved through its dual (Theorem 14.10):

```r
suppressMessages(library(quadprog))
X3 <- rbind(c(3, 4), c(2, 2), c(4, 4), c(1, 4), c(2, 1), c(4, 3), c(4, 1))
y3 <- c(1, 1, 1, 1, -1, -1, -1)
s3 <- solve.QP((y3 %o% y3) * tcrossprod(X3) + diag(1e-9, 7), rep(1, 7),
               cbind(y3, diag(7)), rep(0, 8), meq = 1)
a3 <- pmax(s3$solution, 0); w3 <- drop(t(X3) %*% (a3 * y3)); sv3 <- which(a3 > 1e-6)
b3 <- mean(y3[sv3] - X3[sv3, ] %*% w3)
list(w = w3, b = b3, support_vectors = sv3, margin = 1 / sqrt(sum(w3^2)))
```

```
## $w
## [1] -2  2
##
## $b
## [1] 1
##
## $support_vectors
## [1] 2 3 5 6
##
## $margin
## [1] 0.3536
```

(a)–(c) $\mathbf{w} = (-2, 2)$ and $b = 1$, so the hyperplane is $-2X_1 + 2X_2 + 1 = 0$, equivalently $0.5 - X_1 + X_2 = 0$: classify red if $0.5 - X_1 + X_2 > 0$ and blue otherwise. (d) The distance from the hyperplane to the nearest points is $1/|\mathbf{w}| = 1/\sqrt{8} = 0.354$, the margin; the margin boundaries are $X_2 = X_1$ and $X_2 = X_1 - 1$. (e) The support vectors are observations 2, 3, 5 and 6, the points $(2, 2)$, $(4, 4)$, $(2, 1)$, $(4, 3)$ on those two lines. (f) Observation 7, $(4, 1)$, has $\alpha_7 = 0$ and lies strictly outside the margin, so by Corollary 14.11 and the remark after it the classifier does not depend on it, and a small movement that keeps it outside the margin changes nothing, as for the uncircled points of Figure 14.1, left. (g) For example $X_2 = X_1 - 0.3$, that is $0.3 - X_1 + X_2 = 0$, which separates the classes with a smaller margin. (h) A blue point at $(3, 4.5)$, say, lies among the reds, and no line then separates the classes: the red $(3, 4)$ lies inside the triangle of the blues $(2, 1)$, $(4, 3)$, $(3, 4.5)$, so every half-plane containing those blues contains it. ■

Exercise 14.4 (ISLR §9.7, exercise 4)**.** Simulate 100 observations in two classes with a visible but non-linear boundary. Show that a polynomial or radial kernel beats a support vector classifier on the training data. Which is best on test data?

Solution. Class `out` outside the circle $x_1^2 + x_2^2 = 1.4$, 70 observations for training:

```
set.seed(1); x4 <- matrix(rnorm(200), 100); d4 <- data.frame(x = x4,
  y = factor(ifelse(x4[, 1]^2 + x4[, 2]^2 > 1.4, "out", "in")))
tr4 <- sample(100, 70)
er4 <- function(m, i) mean(predict(m, d4[i, ]) != d4$y[i])
f4 <- list(linear = svm(y ~ ., d4[tr4, ], kernel = "linear", cost = 1),
           poly2 = svm(y ~ ., d4[tr4, ], kernel = "polynomial", degree = 2, cost =
            ↪  1),
           radial = svm(y ~ ., d4[tr4, ], kernel = "radial", gamma = 1, cost = 1))
round(sapply(f4, function(m) c(train = er4(m, tr4), test = er4(m, -tr4))), 3)
```

```
##       linear poly2 radial
## train  0.343 0.086  0.029
## test   0.433 0.033  0.167
```

The support vector classifier cannot follow a circle and misclassifies a third of the training data. Both kernels do far better on the training data, the radial one best; on the test data the quadratic kernel, which matches the true boundary, is best, and the radial kernel, with $\gamma = 1$ fixed, overfits a little. ■

Exercise 14.5 (ISLR §9.7, exercise 5)**.** With $n = 500$, x_1, x_2 uniform on $(-\frac{1}{2}, \frac{1}{2})$ and $y = 1$ when $x_1^2 - x_2^2 > 0$: fit logistic regression in x_1, x_2; then with non-linear functions of them; then a support vector classifier and a support vector machine with a non-linear kernel; compare the training predictions.

Solution.

```
set.seed(1); x1 <- runif(500) - 0.5; x2 <- runif(500) - 0.5
y5 <- 1 * (x1^2 - x2^2 > 0)
d5 <- data.frame(x1, x2, y = factor(y5)); acc5 <- function(p) mean(p == y5)
g5 <- suppressWarnings(glm(y ~ poly(x1, 2) + poly(x2, 2) + I(x1 * x2), d5,
                           family = binomial))
round(c(logistic_linear = acc5(predict(glm(y ~ x1 + x2, d5, family = binomial),
                                       type = "response") > 0.5),
        logistic_quadratic = acc5(predict(g5, type = "response") > 0.5),
        svc = acc5(predict(svm(y ~ ., d5, kernel = "linear", cost = 1)) == "1"),
```

```
    svm_radial = acc5(predict(svm(y ~ ., d5, kernel = "radial", gamma = 1,
                                  cost = 1)) == "1")), 3)
```

```
##    logistic_linear logistic_quadratic                svc
##              0.570               1.000              0.522
##         svm_radial
##              0.972
```

The two linear fits have a linear boundary and do little better than guessing (57% and 52% correct). Logistic regression on the squares and the product classifies every training observation correctly: the boundary $x_1^2 = x_2^2$ is linear in the enlarged features, and the classes are separable there, so the fit warns that fitted probabilities are 0 or 1. The radial support vector machine gets 97% without being told the features. Logistic regression with the right features and a kernel machine reach the same place, as the remark on ISLR §9.5 says. ■

Exercise 14.6 (ISLR §9.7, exercise 6)**.** Generate two-class data with $p = 2$ that are barely linearly separable. Compute the cross-validation error and the number of training errors for a range of `cost`; then the test errors on new data. Which `cost` is best on test data, and how does it compare with the best on training and cross-validation errors?

Solution. Points uniform on the unit square, the class given by the side of the diagonal, with a strip of half-width 0.02 about the diagonal removed; 1000 test points on the whole square:

```
set.seed(1); x6 <- matrix(runif(200), 100); y6 <- ifelse(x6[, 2] > x6[, 1], 1, -1)
k6 <- abs(x6[, 2] - x6[, 1]) > 0.02
d6 <- data.frame(x = x6[k6, ], y = factor(y6[k6]))
cs6 <- c(0.01, 0.1, 1, 10, 100, 1000); set.seed(1)
tu6 <- tune(svm, y ~ ., data = d6, kernel = "linear", ranges = list(cost = cs6))
xt6 <- matrix(runif(2000), 1000)
dt6 <- data.frame(x = xt6, y = factor(ifelse(xt6[, 2] > xt6[, 1], 1, -1)))
round(t(sapply(cs6, function(C) { m <- svm(y ~ ., d6, kernel = "linear", cost = C)
  c(cost = C, training_errors = sum(predict(m) != d6$y),
```

```
    cv_error = tu6$performances$error[tu6$performances$cost == C],
    test_errors = sum(predict(m, dt6) != dt6$y)) })), 3)
```

```
##          cost training_errors cv_error test_errors
## [1,]    0.01               10    0.137         146
## [2,]    0.10                1    0.030          64
## [3,]    1.00                0    0.020          40
## [4,]   10.00                0    0.000           9
## [5,]  100.00                0    0.000           0
## [6,] 1000.00                0    0.000           0
```

Training errors fall to 0 from `cost` 1 on, and the cross-validation error to 0 from 10. Here the test errors keep falling to 0 at the largest values, because the true boundary is exactly linear and the data have no noise: the claim that a smaller `cost`, misclassifying a few training points, can do better on test data needs noise or a boundary the training data mislead about, and this design has neither. With label noise near the boundary the ordering would reverse; the exercise shows that the conclusion depends on the data. ■

Exercise 14.7 (ISLR §9.7, exercise 7)**.** For `Auto`, predict whether a car's mileage is above the median: (a) create the binary response; (b) support vector classifiers over `cost`, by cross-validation; (c) support vector machines with radial and polynomial kernels over `gamma`, `degree` and `cost`.

Solution.

```
Au <- Auto; Au$high <- factor(Au$mpg > median(Au$mpg))
Au <- Au[, !(names(Au) %in% c("mpg", "name"))]; set.seed(1)
t7l <- tune(svm, high ~ ., data = Au, kernel = "linear",
            ranges = list(cost = c(0.01, 0.1, 1, 10, 100)))
set.seed(1)
t7r <- tune(svm, high ~ ., data = Au, kernel = "radial",
            ranges = list(cost = c(0.1, 1, 10, 100), gamma = c(0.01, 0.1, 1)))
set.seed(1)
t7p <- tune(svm, high ~ ., data = Au, kernel = "polynomial",
```

```
            ranges = list(cost = c(0.1, 1, 10, 100), degree = 2:3))
round(t7l$performances[, 1:2], 4)
```

```
##     cost  error
## 1   0.01 0.0892
## 2   0.10 0.0919
## 3   1.00 0.0844
## 4  10.00 0.0895
## 5 100.00 0.0869
```

```
rbind(radial = c(unlist(t7r$best.parameters), cv_error = t7r$best.performance),
      polynomial = c(unlist(t7p$best.parameters), cv_error = t7p$best.performance))
```

```
##             cost gamma cv_error
## radial         1     1  0.06635
## polynomial   100     3  0.08423
```

(a) The response is **high** in the code. (b) The linear classifier's cross-validation error is between 8.4% and 9.2% for every **cost** from 0.01 to 100, smallest at 1: the classes are nearly linearly separable and the penalty hardly matters. (c) The radial kernel with **cost** 1 and $\gamma = 1$ does best, about 6.6%; the best polynomial kernel, degree 3 (printed under the header **gamma**), does about

the same as the linear one. A slightly non-linear boundary helps a little; the differences are within the noise of ten-fold cross-validation on 392 cars. ■

Exercise 14.8 (ISLR §9.7, exercise 8)**.** For `OJ`: (a) draw a training set of 800 observations; (b), (c) a support vector classifier with `cost` 0.01, its support vectors and its training and test error rates; (d), (e) the same with `cost` chosen by cross-validation over 0.01 to 10; (f), (g) the same with a radial kernel and with a polynomial kernel of degree 2; (h) which is best?

Solution.

```
set.seed(1); tro <- sample(nrow(OJ), 800)
oe <- function(m) c(train = mean(predict(m, OJ[tro, ]) != OJ$Purchase[tro]),
                    test = mean(predict(m, OJ[-tro, ]) != OJ$Purchase[-tro]))
m8 <- svm(Purchase ~ ., OJ[tro, ], kernel = "linear", cost = 0.01)
c(support_vectors = m8$tot.nSV)
```

```
## support_vectors
##             435
```

```
t(sapply(c("linear", "radial", "polynomial"), function(k) { set.seed(1)
  tk <- tune(svm, Purchase ~ ., data = OJ[tro, ], kernel = k, degree = 2,
```

```
            ranges = list(cost = 10^seq(-2, 1, by = 0.5)))
b <- tk$best.parameters$cost
round(c(best_cost = b,
        oe(svm(Purchase ~ ., OJ[tro, ], kernel = k, degree = 2, cost = 0.01)),
        oe(svm(Purchase ~ ., OJ[tro, ], kernel = k, degree = 2, cost = b))), 4)
```

```
##            best_cost  train   test  train   test
## linear         3.162 0.1750 0.1778 0.1650 0.1519
## radial         1.000 0.3938 0.3778 0.1512 0.1852
## polynomial     3.162 0.3725 0.3667 0.1550 0.2037
```

(a) The training set is `tro` in the code. (b), (c) With `cost` 0.01 the classifier keeps 435 of the 800 training observations as support vectors, a very wide margin, and errs on 17.5% of the training and 17.8% of the test observations. (d), (e) Cross-validation chooses `cost` 3.16, with errors 16.5% and 15.2%. (f), (g) At `cost` 0.01 the radial and polynomial machines classify almost everything as the majority brand, about 38% error; tuned, they reach test errors of 18.5% and 20.4%. (h) The tuned linear classifier is best on this split: the extra flexibility of the kernels buys nothing here. ■

Exercise 14.9 (ISLR §10.10, exercise 1)**.** A network has $p = 4$ inputs, two hidden layers of 2 and 3 units, and one output. (a) Draw it; (b) write $f(X)$ with rectified units; (c) evaluate it for some coefficients; (d) count the parameters.

Solution. (a) Four input nodes, each joined to both units of the first layer; each of those joined to all three units of the second layer; those three joined to the output. (b) With $g(z) = \max(0, z)$,

$$A_k^{(1)} = g\Big(w_{k0}^{(1)} + \sum_{j=1}^{4} w_{kj}^{(1)} X_j\Big), \quad k = 1, 2;$$

$$A_l^{(2)} = g\Big(w_{l0}^{(2)} + \sum_{k=1}^{2} w_{lk}^{(2)} A_k^{(1)}\Big), \quad l = 1, 2, 3;$$

$$f(X) = \beta_0 + \sum_{l=1}^{3} \beta_l A_l^{(2)}.$$

(c) For instance with every weight 1, every offset 0 and $X = (1, -1, 2, 0)$: $A_k^{(1)} = g(2) = 2$, $A_l^{(2)} = g(4) = 4$ and $f(X) = 12$. (d) $2(4+1) + 3(2+1) + (3+1) = 10 + 9 + 4 = 23$ parameters, the count of Definition 14.17 layer by layer. ■

Exercise 14.10 (ISLR §10.10, exercise 2)**.** (a) Show that adding a constant c to every Z_ℓ in the softmax (14) leaves the probabilities unchanged. (b) In the multinomial logistic model, show that adding constants c_j to the jth coefficient of every class leaves the predictions unchanged.

Solution. (a) $e^{Z_m+c}/\sum_\ell e^{Z_\ell+c} = e^c e^{Z_m}/(e^c \sum_\ell e^{Z_\ell}) = f_m$. (b) The multinomial logistic model is (14) with $Z_m = \beta_{m0} + \sum_j \beta_{mj} x_j$. Adding c_j to β_{mj} for every m adds $c_0 + \sum_j c_j x_j$ to every Z_m, the same amount for each class at a given $\mathbf{x}$, so by (a) the probabilities, and hence the predictions, are unchanged. The parameters are therefore not identified; one class is usually fixed as the baseline (Unit 7). ■

Exercise 14.11 (ISLR §10.10, exercise 3)**.** Show that the cross-entropy of Definition 14.18 with $M = 2$ classes is the negative log of the logistic regression likelihood.

Solution. With two classes $y_{i1} = y_i$, $y_{i0} = 1 - y_i$ and, by Exercise 14.10(a), we may take $Z_0 = 0$, so $f_1 = e^{Z_1}/(1 + e^{Z_1}) = p(\mathbf{x})$ is the logistic function of Z_1 and $f_0 = 1 - p(\mathbf{x})$. The cross-entropy is then $-\sum_i \{y_i \ln p(\mathbf{x}_i) + (1 - y_i) \ln(1 - p(\mathbf{x}_i))\} = -\ln \prod_i p(\mathbf{x}_i)^{y_i} \{1 - p(\mathbf{x}_i)\}^{1-y_i}$, the negative log of the likelihood of the logistic model, ISLR's (4.5). ■

Exercise 14.12 (ISLR §10.10, exercise 4)**.** A convolutional network takes 32×32 greyscale images and has one convolution layer with three 5×5 filters, without padding. (ISLR's part (a), a sketch of the network, is omitted.) (b) How many parameters? (c) Explain it as an ordinary feed-forward network with constrained weights. (d) How many weights without the constraints?

Solution. (b) Each filter has 25 weights and a bias, so $3 \times 26 = 78$ parameters (Definition 14.19). Each feature map is $(32 - 5 + 1)^2 = 28^2 = 784$ units. (c) Each of the $3 \times 784 = 2352$ hidden units is an ordinary unit whose weights on the 1024 pixels are zero outside its 5×5 patch, and the 25 non-zero weights and the bias are the same for every unit of one feature map. (d) Without the constraints each hidden unit has 1024 weights and a bias: $2352 \times 1025 = 2{,}410{,}800$. ■

Exercise 14.13 (ISLR §10.10, exercise 5)**.** In ISLR's Table 10.2 the three methods on `Hitters` are ordered differently by mean absolute error (lasso 252.3, least squares 254.7, network 257.4) and by test R^2 (least squares 0.56, network 0.54, lasso 0.51). How can this be?

Solution. Test R^2 is one minus the mean squared error over the variance of the test responses, so it ranks methods by squared error; mean absolute error ranks them by absolute error. Squaring weights the large errors more. A method with smaller typical errors and a few larger ones, the lasso here, which shrinks the predictions of the highest salaries, can have the smaller mean absolute error and the larger mean squared error. Example 14.5 reproduces the pattern. ■

Exercise 14.14 (ISLR §10.10, exercise 6)**.** For $R(\beta) = \sin\beta + \beta/10$: (a) graph it on $[-6, 6]$; (b) find R'; (c) run gradient descent from $\beta^0 = 2.3$ with learning rate 0.1; (d) repeat from $\beta^0 = 1.4$.

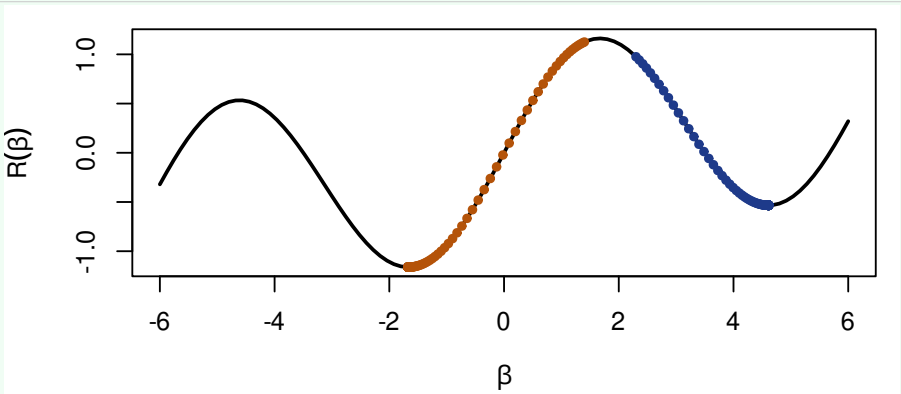
Solution.

```
R6 <- function(b) sin(b) + b / 10; dR6 <- function(b) cos(b) + 0.1
gd6 <- function(b) { path <- b
  repeat { bn <- b - 0.1 * dR6(b); path <- c(path, bn)
    if (abs(bn - b) < 1e-8) break; b <- bn }; path }
```

```r
p23 <- gd6(2.3); p14 <- gd6(1.4)
rbind(from_2.3 = c(limit = tail(p23, 1), steps = length(p23) - 1),
      from_1.4 = c(limit = tail(p14, 1), steps = length(p14) - 1))
```

```
##           limit steps
## from_2.3  4.612   173
## from_1.4 -1.671   183
```

```r
par(mar = c(4, 4.2, 0.8, 0.8), cex = 0.85)
curve(R6, -6, 6, lwd = 2, xlab = expression(beta), ylab = expression(R(beta)))
points(p23, R6(p23), col = "#1E3A8A", pch = 20)
points(p14, R6(p14), col = "#B45309", pch = 20)
```



(b) $R'(\beta) = \cos\beta + 1/10$. (c) From 2.3 the iterates move right and settle at 4.612, the local minimum where $\cos\beta = -0.1$ with $\sin\beta < 0$. (d) From 1.4 they move left, to -1.671, a different local minimum with a lower value. R is not convex, so where gradient descent ends depends on where it starts: the situation of a neural network (Proposition 14.22), which Theorem 14.4 does not cover. ■

Exercise 14.15 (ISLR §10.10, exercise 7)**.** Fit a network with one hidden layer of 10 units and dropout to `Default`, and compare its classification with linear logistic regression.

Solution. A third of the 10,000 customers held out; the network is trained on the cross-entropy of Definition 14.18 for 100 epochs:

```
set.seed(1); yd <- as.numeric(Default$default == "Yes")
ted <- sample(10000, 10000 / 3)
Xd <- scale(cbind(student = as.numeric(Default$student == "Yes"), Default$balance,
                  Default$income))
pl7 <- predict(glm(default ~ ., Default[-ted, ], family = binomial), Default[ted, ],
               type = "response") > 0.5
torch_manual_seed(1)
```

```
nd <- fitnet(nn_sequential(nn_linear(3, 10), nn_relu(), nn_dropout(0.4),
                           nn_linear(10, 1)),
             tf(Xd[-ted, ]), tf(matrix(yd[-ted])), 100, 32,
             nnf_binary_cross_entropy_with_logits)
pn7 <- as.numeric(torch_sigmoid(nd(tf(Xd[ted, ])))) > 0.5
round(c(logistic = mean(pl7 == yd[ted]), network = mean(pn7 == yd[ted]),
        always_no = mean(yd[ted] == 0)), 4)
```

```
##  logistic   network always_no
##    0.9730    0.9709    0.9652
```

Both classify about 97% of the test customers correctly, against 96.5% for predicting that nobody defaults; logistic regression is slightly ahead. With three predictors and a boundary close to linear in `balance`, the network has nothing to add. ■

Exercise 14.16 (ISLR §10.10, exercise 11)**.** Fit a non-linear autoregression to `NYSE` by flattening the five-day sequences of Example 14.6 into fifteen inputs of a feed-forward network, and compare its test R^2 with the linear autoregression on the same inputs.

Solution.

```
Fl <- matrix(Xs, nrow = dim(Xs)[1])
lin11 <- lm(y ~ ., data.frame(y = yv, Fl)[itr, ])
torch_manual_seed(1); set.seed(1)
nl11 <- fitnet(nn_sequential(nn_linear(15, 32), nn_relu(), nn_dropout(0.5),
                             nn_linear(32, 1)),
               tf(Fl[itr, ]), tf(matrix(yv[itr])), 200, 32)
round(c(linear_ar = r2v(predict(lin11, data.frame(Fl)[!itr, ])),
        nonlinear_ar = r2v(as.numeric(nl11(tf(Fl[!itr, ]))))), 3)
```

```
##    linear_ar nonlinear_ar
##        0.413        0.420
```

The linear autoregression on the flattened sequences has test R^2 0.41, Unit 10's value; a network with 32 rectified units reaches about the same, a little higher or lower depending on the run. The relation is close to linear and nothing is gained. ■

Exercise 14.17 (ISLR §10.10, exercise 12)**.** Add `day_of_week` to the recurrent network of Example 14.6 and compute the test R^2.

Solution. The five day indicators are appended to the three series at every step of each sequence:

```
dwk <- model.matrix(~ day_of_week - 1, NYSE); Xs2 <- seqs(cbind(xd, dwk))
torch_manual_seed(1); set.seed(1)
rn2 <- fitnet(rnnmod(8), tf(Xs2[itr, , ]), tf(matrix(yv[itr])), 200, 64)
c(rnn_with_day_test_r2 = r2v(as.numeric(rn2(tf(Xs2[!itr, , ])))))
```

```
## rnn_with_day_test_r2
##               0.4572
```

The test R^2 rises to about 0.46, the gain Unit 10 found by adding the day to the linear autoregression: trading volume has a weekly pattern that the five lagged values do not carry. ■

Exercise 14.18 (Boyd and Vandenberghe, exercise 5.1)**.** Minimise x^2+1 subject to $(x-2)(x-4) \leqslant 0$. (a) Give the feasible set, the optimal value and point. (b) Find the dual function g. (c) State the dual problem, find $\lambda^\star$ and $d^\star$; does strong duality hold? (d) Let $p^\star(u)$ be the optimal value when the constraint is $(x-2)(x-4) \leqslant u$; verify $dp^\star(0)/du = -\lambda^\star$.

Solution. (a) The feasible set is $[2,4]$; the objective increases on it, so $x^\star = 2$ and $p^\star = 5$. (b) $L(x,\lambda) = (1+\lambda)x^2 - 6\lambda x + 1 + 8\lambda$. For $\lambda > -1$ it is minimised at $x = 3\lambda/(1+\lambda)$, so

$g(\lambda) = -9\lambda^2/(1+\lambda)+1+8\lambda$; for $\lambda \leqslant -1$ it is unbounded below and $g = -\infty$. (c) Maximise $g(\lambda)$ over $\lambda \geqslant 0$; g is concave (the prerequisites' proposition **concavity of the dual**), and $g'(\lambda) = 0$ gives $(1+\lambda)^2 = 9$, $\lambda^\star = 2$ and $d^\star = g(2) = -12 + 17 = 5 = p^\star$. Strong duality holds, as it must: $x = 3$ is strictly feasible, so Slater's condition holds. (d) For $-1 \leqslant u \leqslant 8$ the feasible set is $[3 - \sqrt{1+u}, 3 + \sqrt{1+u}]$ and $p^\star(u) = (3 - \sqrt{1+u})^2 + 1 = 11 + u - 6\sqrt{1+u}$, whose derivative at 0 is $1 - 3 = -2 = -\lambda^\star$. ■

Exercise 14.19 (Boyd and Vandenberghe, exercise 5.5)**.** Find the dual of the linear programme: minimise $\mathbf{c}'\mathbf{x}$ subject to $\mathbf{G}\mathbf{x} \leqslant \mathbf{h}$ and $\mathbf{A}\mathbf{x} = \mathbf{b}$, making the implicit constraints explicit.

Solution. Writing $\mathbf{A}\mathbf{x} = \mathbf{b}$ as two inequalities with multipliers $\boldsymbol{\nu}^+, \boldsymbol{\nu}^- \geqslant \mathbf{0}$ and putting $\boldsymbol{\nu} = \boldsymbol{\nu}^+ - \boldsymbol{\nu}^-$, which is then unrestricted in sign, the Lagrangian is $L = \mathbf{c}'\mathbf{x} + \boldsymbol{\lambda}'(\mathbf{G}\mathbf{x} - \mathbf{h}) + \boldsymbol{\nu}'(\mathbf{A}\mathbf{x} - \mathbf{b}) = (\mathbf{c} + \mathbf{G}'\boldsymbol{\lambda} + \mathbf{A}'\boldsymbol{\nu})'\mathbf{x} - \mathbf{h}'\boldsymbol{\lambda} - \mathbf{b}'\boldsymbol{\nu}$, affine in $\mathbf{x}$. Its infimum is $-\mathbf{h}'\boldsymbol{\lambda} - \mathbf{b}'\boldsymbol{\nu}$ if $\mathbf{c} + \mathbf{G}'\boldsymbol{\lambda} + \mathbf{A}'\boldsymbol{\nu} = \mathbf{0}$ and $-\infty$ otherwise. The dual is: maximise $-\mathbf{h}'\boldsymbol{\lambda} - \mathbf{b}'\boldsymbol{\nu}$ subject to $\mathbf{c} + \mathbf{G}'\boldsymbol{\lambda} + \mathbf{A}'\boldsymbol{\nu} = \mathbf{0}$ and $\boldsymbol{\lambda} \geqslant \mathbf{0}$, again a linear programme. ■

Exercise 14.20 (Boyd and Vandenberghe, exercise 5.27)**.** Minimise $|\mathbf{Ax}-\mathbf{b}|^2$ subject to $\mathbf{Gx}=\mathbf{h}$, where $\mathbf{A}\in\mathbb{R}^{m\times n}$ has rank n and $\mathbf{G}\in\mathbb{R}^{p\times n}$ rank p. Give the Karush–Kuhn–Tucker conditions and solve them for $\mathbf{x}^\star$ and the multiplier $\boldsymbol{\nu}^\star$.

Solution. The conditions are $2\mathbf{A}'(\mathbf{Ax}^\star-\mathbf{b})+\mathbf{G}'\boldsymbol{\nu}^\star=\mathbf{0}$ and $\mathbf{Gx}^\star=\mathbf{h}$ (stationarity of $L=|\mathbf{Ax}-\mathbf{b}|^2+\boldsymbol{\nu}'(\mathbf{Gx}-\mathbf{h})$ and feasibility). With each equality written as two inequalities with multipliers $\boldsymbol{\nu}^+,\boldsymbol{\nu}^-\geqslant\mathbf{0}$ and $\boldsymbol{\nu}=\boldsymbol{\nu}^+-\boldsymbol{\nu}^-$, the problem is convex, so by the converse half of the prerequisites' theorem **Karush–Kuhn–Tucker** a solution of these conditions is optimal; the necessity half is not needed, and Slater's condition fails for the split equalities. The first gives $\mathbf{x}^\star=(\mathbf{A}'\mathbf{A})^{-1}(\mathbf{A}'\mathbf{b}-\frac{1}{2}\mathbf{G}'\boldsymbol{\nu}^\star)$, and substituting into the second,

$$\boldsymbol{\nu}^\star=-2\{\mathbf{G}(\mathbf{A}'\mathbf{A})^{-1}\mathbf{G}'\}^{-1}\{\mathbf{h}-\mathbf{G}(\mathbf{A}'\mathbf{A})^{-1}\mathbf{A}'\mathbf{b}\},$$

the inverse existing because $\mathbf{G}$ has full row rank and $(\mathbf{A}'\mathbf{A})^{-1}$ is positive definite. So $\mathbf{x}^\star$ is the least squares estimate corrected towards the constraint, the restricted least squares estimator of Unit 3.

```
set.seed(1); A <- matrix(rnorm(40), 10); bb <- rnorm(10)
G <- matrix(rnorm(8), 2); h <- rnorm(2)
```

```
AtA <- solve(crossprod(A))
nu <- -2 * solve(G %*% AtA %*% t(G), h - G %*% AtA %*% crossprod(A, bb))
xs <- AtA %*% (crossprod(A, bb) - t(G) %*% nu / 2)
c(constraint_residual = max(abs(G %*% xs - h)),
  stationarity_residual = max(abs(2 * t(A) %*% (A %*% xs - bb) + t(G) %*% nu)))
```

```
##   constraint_residual stationarity_residual
##             7.772e-16              2.220e-15
```

Exercise 14.21 (Boyd and Vandenberghe, exercise 7.3)**.** Let $y = 1$ if $\mathbf{a}'\mathbf{u} + b + v \leqslant 0$ and $y = 0$ otherwise, with v standard normal. Formulate maximum likelihood estimation of $(\mathbf{a}, b)$ from data $(\mathbf{u}_i, y_i)$ as a convex problem.

Solution. $\Pr(y = 1) = \Phi(-\mathbf{a}'\mathbf{u} - b)$ and $\Pr(y = 0) = \Phi(\mathbf{a}'\mathbf{u} + b)$, so the log likelihood is $\sum_{y_i=1} \ln \Phi(-\mathbf{a}'\mathbf{u}_i - b) + \sum_{y_i=0} \ln \Phi(\mathbf{a}'\mathbf{u}_i + b)$. The normal distribution function Φ is log-concave: with $\phi = \Phi'$ and $\phi'(x) = -x\phi(x)$, $(\ln \Phi)'' = -\phi(x)\{x\Phi(x) + \phi(x)\}/\Phi(x)^2$, and $x\Phi(x) + \phi(x)$ has derivative $\Phi(x) > 0$ and limit 0 as $x \to -\infty$ (for $x < 0$, $\Phi(x) \leqslant \int_{-\infty}^{x} (t/x)\phi(t)\,dt = \phi(x)/|x|$, so $|x\Phi(x)| \leqslant \phi(x) \to 0$), so it is positive and $(\ln \Phi)'' < 0$. And $\ln \Phi$ composed with an affine function

of $(\mathbf{a}, b)$ is concave, so the log likelihood is concave and its maximisation, the minimisation of its negative, is a convex problem without constraints. This is the probit model of Unit 7 with the sign of the coefficients reversed. ■

Exercise 14.22 (Boyd and Vandenberghe, exercise 8.23)**.** Points $\mathbf{x}_1, \dots, \mathbf{x}_N$ and $\mathbf{y}_1, \dots, \mathbf{y}_M$ are to be separated by maximising t subject to $\mathbf{a}'\mathbf{x}_i - b \geqslant t$, $\mathbf{a}'\mathbf{y}_i - b \leqslant -t$ and $|\mathbf{a}| \leqslant 1$. (a) Show $t^\star > 0$ if and only if the points are linearly separable, and then $|\mathbf{a}^\star| = 1$. (b) Show that with $\tilde{\mathbf{a}} = \mathbf{a}/t$, $\tilde{b} = b/t$ the problem is equivalent to minimising $|\tilde{\mathbf{a}}|$ subject to $\tilde{\mathbf{a}}'\mathbf{x}_i - \tilde{b} \geqslant 1$ and $\tilde{\mathbf{a}}'\mathbf{y}_i - \tilde{b} \leqslant -1$.

Solution. (a) If $t^\star > 0$ then $\mathbf{a}^{\star\prime}\mathbf{x}_i \geqslant b^\star + t^\star > b^\star - t^\star \geqslant \mathbf{a}^{\star\prime}\mathbf{y}_j$, so $\{\mathbf{z} : \mathbf{a}^{\star\prime}\mathbf{z} = b^\star\}$ separates. Conversely a separating $(\mathbf{a}, b)$, scaled so $|\mathbf{a}| \leqslant 1$ and shifted to the middle of the gap, gives a feasible point with $t > 0$. If $|\mathbf{a}^\star| < 1$, then $(\mathbf{a}^\star, b^\star, t^\star)/|\mathbf{a}^\star|$ is feasible with a larger t, so $|\mathbf{a}^\star| = 1$. (b) A feasible $(\mathbf{a}, b, t)$ with $t > 0$ gives a feasible $(\tilde{\mathbf{a}}, \tilde{b})$ with $|\tilde{\mathbf{a}}| = |\mathbf{a}|/t \leqslant 1/t$; conversely a feasible $(\tilde{\mathbf{a}}, \tilde{b})$ gives $t = 1/|\tilde{\mathbf{a}}|$ with $(\tilde{\mathbf{a}}, \tilde{b})/|\tilde{\mathbf{a}}|$. So maximising t is minimising $|\tilde{\mathbf{a}}|$, and the transformed problem is the maximal margin problem (5) with $\mathbf{w} = \tilde{\mathbf{a}}$, offset $-\tilde{b}$ and labels ± 1 for the two sets: $t^\star$ is the margin of Theorem 14.8. ■

Exercise 14.23 (Boyd and Vandenberghe, exercise 9.1)**.** Minimise $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}'\mathbf{P}\mathbf{x} + \mathbf{q}'\mathbf{x} + r$ with $\mathbf{P}$ symmetric. (a) Show that if $\mathbf{P}$ is not positive semi-definite the problem is unbounded below. (b) Show that if $\mathbf{P}$ is positive semi-definite but $\mathbf{P}\mathbf{x} = -\mathbf{q}$ has no solution, it is unbounded below.

Solution. (a) Take $\mathbf{v}$ with $\mathbf{v}'\mathbf{P}\mathbf{v} < 0$; along $\mathbf{x} = s\mathbf{v}$, $f = \frac{1}{2}s^2\mathbf{v}'\mathbf{P}\mathbf{v} + s\mathbf{q}'\mathbf{v} + r \to -\infty$. (b) $\mathbf{q}$ is not in the column space of $\mathbf{P}$; let $\mathbf{v} \neq \mathbf{0}$ be its component orthogonal to that space (the prerequisites' theorem **orthogonal projection**). Then $\mathbf{P}\mathbf{v} = \mathbf{0}$, because $\mathbf{P}$ is symmetric and $\mathbf{v}$ is orthogonal to its columns, and $\mathbf{q}'\mathbf{v} = |\mathbf{v}|^2 > 0$. Along $\mathbf{x} = -s\mathbf{v}$, $f = -s|\mathbf{v}|^2 + r \to -\infty$. Conversely, if $\mathbf{P}$ is positive semi-definite and $\mathbf{P}\mathbf{x}^\star = -\mathbf{q}$, then f is convex and $\nabla f(\mathbf{x}^\star) = \mathbf{0}$, so $\mathbf{x}^\star$ is a minimiser (the prerequisites' proposition **first-order condition**). So a convex quadratic has a minimiser exactly when its stationarity equation is solvable, the normal equations of least squares being the case $\mathbf{P} = 2\mathbf{X}'\mathbf{X}$. ■

Exercise 14.24 (Boyd and Vandenberghe, exercise 9.6)**.** For $f(\mathbf{x}) = \frac{1}{2}(x_1^2 + \gamma x_2^2)$, $\gamma > 0$, gradient descent with exact line search from $\mathbf{x}^{(0)} = (\gamma, 1)$: verify that $\mathbf{x}^{(k)} = \{(\gamma - 1)/(\gamma + 1)\}^k(\gamma, (-1)^k)$.

Solution. Suppose $\mathbf{x}^{(k)} = c^k(\gamma, (-1)^k)$ with $c = (\gamma-1)/(\gamma+1)$; it holds at $k=0$. The gradient is $(x_1, \gamma x_2) = c^k\gamma(1, (-1)^k)$, so $\mathbf{x}^{(k)} - t\nabla f = c^k(\gamma(1-t), (-1)^k(1-\gamma t))$ and

$$\phi(t) = f(\mathbf{x}^{(k)} - t\nabla f) = \tfrac{1}{2}c^{2k}\{\gamma^2(1-t)^2 + \gamma(1-\gamma t)^2\}.$$

ϕ is a convex quadratic with $\phi'(t) = -c^{2k}\gamma^2\{(1-t)+(1-\gamma t)\}$, which vanishes at $t = 2/(1+\gamma)$. There $1-t = c$ and $1-\gamma t = -c$, so $\mathbf{x}^{(k+1)} = c^k(\gamma c, -(-1)^k c) = c^{k+1}(\gamma, (-1)^{k+1})$, and induction completes the proof. Each step multiplies f by c^2: with condition number γ (or $1/\gamma$) the rate is $\{(\gamma-1)/(\gamma+1)\}^2$, close to 1 when γ is far from 1, the zigzag of the remark after Theorem 14.4. ■

Exercise 14.25 (Boyd and Vandenberghe, exercise 9.30)**.** Minimise $f(\mathbf{x}) = -\sum_{i=1}^m \ln(1-\mathbf{a}_i'\mathbf{x}) - \sum_{j=1}^n \ln(1-x_j^2)$, the analytic centre of the inequalities $\mathbf{a}_i'\mathbf{x} \leqslant 1$ and $|x_j| \leqslant 1$, from $\mathbf{x}^{(0)} = \mathbf{0}$: (a) by gradient descent with backtracking, stopping when $|\nabla f| \leqslant 10^{-3}$, varying the backtracking parameters; (b) by Newton's method, stopping when $\lambda^2/2 \leqslant 10^{-10}$, $\lambda^2 = \nabla f(\mathbf{x})'\{\nabla^2 f(\mathbf{x})\}^{-1}\nabla f(\mathbf{x})$ the (squared) Newton decrement.

Solution. $\nabla f = \sum_i \mathbf{a}_i/(1-\mathbf{a}_i'\mathbf{x}) + (2x_j/(1-x_j^2))_j$ and $\nabla^2 f = \sum_i \mathbf{a}_i\mathbf{a}_i'/(1-\mathbf{a}_i'\mathbf{x})^2 + \mathrm{diag}\{2(1+x_j^2)/(1-x_j^2)^2\}$ by the chain rule. With $m = 200$, $n = 100$ and standard normal $\mathbf{a}_i$:

```
set.seed(1); m30 <- 200; n30 <- 100; A30 <- matrix(rnorm(m30 * n30), m30)
f30 <- function(x) -sum(log(1 - A30 %*% x)) - sum(log(1 - x^2))
g30 <- function(x) drop(t(A30) %*% (1 / (1 - A30 %*% x))) + 2 * x / (1 - x^2)
h30 <- function(x) { d <- drop(1 / (1 - A30 %*% x))^2
  t(A30) %*% (A30 * d) + diag(2 * (1 + x^2) / (1 - x^2)^2) }
ok30 <- function(x) all(A30 %*% x < 1) && all(abs(x) < 1)
run30 <- function(newton, al = 0.01, be = 0.5) { x <- rep(0, n30); vals <- f30(x)
  for (k in 1:5000) { g <- g30(x); v <- if (newton) -solve(h30(x), g) else -g
    done <- if (newton) -sum(g * v) / 2 <= 1e-10 else sqrt(sum(g^2)) <= 1e-3
    if (done) break
    t <- 1; while (!ok30(x + t * v)) t <- be * t
    while (f30(x + t * v) > f30(x) + al * t * sum(g * v)) t <- be * t
    x <- x + t * v; vals <- c(vals, f30(x)) }
  list(iterations = k - 1, value = f30(x), path = vals) }
nt30 <- run30(TRUE)
c(gradient_iterations = run30(FALSE)$iterations,
  gradient_alpha_0.3 = run30(FALSE, al = 0.3)$iterations,
```

```
  gradient_beta_0.8 = run30(FALSE, be = 0.8)$iterations,
  newton_iterations = nt30$iterations, p_star = nt30$value)
```

```
## gradient_iterations  gradient_alpha_0.3   gradient_beta_0.8
##             1003.00              796.00             1594.00
##   newton_iterations              p_star
##                7.00              -77.93
```

```
signif(nt30$path - nt30$value, 3)
```

```
## [1] 7.79e+01 3.13e+01 6.07e+00 2.02e-01 7.53e-03 8.66e-05
## [7] 1.36e-08 0.00e+00
```

(a) Gradient descent needs about a thousand iterations with $\alpha = 0.01$, $\beta = 0.5$, and about 800 with $\alpha = 0.3$, which accepts only steps that decrease f by more; with $\alpha = 0.01$ and $\beta = 0.8$ it needs about 1600. The Hessian is badly conditioned, and the convergence is linear, the behaviour Theorem 14.4 proves under global bounds m, M (which fail here: the Hessian is unbounded near the boundary of the domain, and the steps come from backtracking, not $t = 1/M$). (b) Newton's method stops after 7 iterations. Its gap to the optimum falls from 31 to 6 to 0.2 to 0.008 to

10^{-4} to 10^{-8}: the number of correct digits roughly doubles at each of the last steps, the quadratic convergence of the remark after Theorem 14.4. ■